

68000 · Z80 · YM2610

NEO GEO SDK

Programmer's Manual and Library Reference

Eagle Software

Release v1.7.0 · MIT

<https://github.com/eaglesoftware777/neogeosdk>

SNK Neo Geo · MVS arcade and AES home · cartridge-era hardware

Contents

Part 1 · Introduction

docs/INTRODUCTION.md

1. What this SDK is
2. The target machine
3. What is in the box
4. The two engines
5. Where to go next
6. Licence and intended use

Part 2 · Programmer's Manual

docs/PROGRAMMERS_MANUAL.md

Part I — The build

1. Two makefiles, one build
2. Selecting a game
3. The five stages
4. Build behaviour worth knowing
5. Debugging

Part II — Writing the game

6. The contract
7. Project layout
8. Driving the engine
9. Characters
10. Sprites, slots, and priority
11. The FIX layer
12. Palettes and the backdrop
13. Input

Part III — Assets

14. Art
15. Sound

Part IV — Staying inside the frame

16. The budget
17. The rules

Appendix A — Register quick reference

Appendix B — Interrupts

Appendix C — Common symptoms

Part 3 · Tutorial: Hello World

docs/HELLO_WORLD.md

0. Before you start
1. Build and run it
2. What actually ran
3. Change something
4. Add a sprite
5. Turn it into your own game
6. Stepping up to the engine
7. When it does not work

Part 4 · SDK Library Reference

SDK_API_GUIDE.md

Sprite Control Block Helpers

Palette and VRAM Helpers
FIX Layer and HUD Helpers
Timing and Housekeeping Helpers
Input and Status Helpers
Sound API
2D Game Engine Layer
Typical Call Patterns
Notes

Part 5 • 2D Engine Reference — C

docs/API_2D_ENGINE_C.md

Contents

Core

ng_defs
ng_engine
ng_game_events
ng_game_interrupt
ng_game_time
ng_progress
ng_properties
ng_runtime
ng_scene
ng_status
ng_timers

Math

ng_fixed

Video

ng_art_asset
ng_bg
ng_depthfx
ng_fix
ng_palette_assets
ng_palette_fx
ng_render_queue
ng_sprite_group
ng_sprite_pool
ng_sprite_window
ng_vram

Characters

ng_actions
ng_border_constraints
ng_chars
ng_joystick
ng_npcs
ng_physics

World

ng_camera
ng_level

Effects

ng_debug
ng_demo_advanced
ng_feedback
ng_particles

Part 6 · 2D Engine Reference — C++

`docs/API_2D_ENGINE_CPP.md`

What this build adds
Choosing between the builds
Classes and methods
Free functions

Part 7 · Sound Driver Reference

`docs/SOUND_DRIVER.md`

1. Architecture
2. The 68000-side API
3. The raw command protocol
4. Content pipelines
5. Practices that keep audio stable
6. What the demo shows

Part 8 · Graphics Pipeline Reference

`docs/ARTBOX_PIPELINE.md`

The colour target
The v1.7.0 quantiser
Alternative pipelines
Asset ordering and ids
Generated outputs
Verifying a build
Artbox Studio
Inputs
Rule Fields
Categories
Generated Files
Build
Runtime Use
Artbox Studio UI

Part 9 · Tools and Utilities

`docs/TOOLS.md`

1. Graphics — `artbox/`
 - 1.1 Conversion cores
 - 1.2 Pipeline stages
 - 1.3 Utilities
 - 1.4 Shell wrappers
 - 1.5 GIMP plug-ins — `tools/gimp-plugins/`
2. Sound — `sound/tools/`
 - 2.1 Compilers
 - 2.2 Sample encoding
 - 2.3 Driver build and comparison
 - 2.4 Sound Studio
3. Verification — `tools/`
4. Launchers and environment
 - 4.1 Repository root
 - 4.2 `dist/`
 - 4.3 `install/`
 - 4.4 `win/`
5. Desktop applications
 - 5.1 Artbox Studio — `artbox/artbox_studio.py`

5.2 Sound Studio — `sound/sound_studio.py`

6. Debug artefacts

PART 1

Introduction

What the SDK is, the machine it targets, and how to read the rest.

`docs/INTRODUCTION.md`

Eagle Software · v1.7.0 · MIT

This is the document to read first. It explains what the SDK is, what the machine underneath it looks like, what you get in the box, and the order in which the rest of the documentation is meant to be read.

1. What this SDK is

The Neo Geo SDK is a hardware-centered development kit for SNK Neo Geo cartridge hardware — the MVS arcade board and the AES home console. It gives you a 68000 cross-build, a Z80 sound stack, two asset pipelines, a 2D game engine in both C and C++, six working example games, and an emulator test loop that runs from make.

It deliberately does **not** hide the hardware.

On a Neo Geo, graphics, palettes, FIX tiles, ROM layout, and sound commands are part of day-to-day programming rather than something an engine abstracts away. That is the machine's actual programming model, and it is the reason Neo Geo software looks and performs the way it does. The SDK preserves that model and modernises everything around it: the build, the asset conversion, the debugging, and the test cycle.

The SDK preserves	The SDK modernises	The SDK resists
Direct register access, real VRAM/palette writes, hardware sprite budgets	Build tooling on Linux, WSL and Windows; Python asset pipelines; MAME-driven test loops; GDB	Heavy abstraction layers that hide what the hardware is doing

2. The target machine

Three chips, one bus, no operating system.

Part	Chip	Role
Main CPU	Motorola 68000 @ 12 MHz	Game logic, VRAM, palettes, sprites, DMA, FIX tiles
Sound CPU	Zilog Z80A @ 4 MHz	Runs the sound driver; talks to the YM2610 over command ports
Sound chip	Yamaha YM2610	FM synthesis, SSG square waves, ADPCM-A and ADPCM-B sample playback

How a frame is drawn

A Neo Geo frame is **stacked, not composited**. Three fixed layers, fixed priority:

- FIX layer** — a 40×32 cell grid of 8×8 tiles, of which **28 rows are visible**, in a fixed screen position. 15 colours per cell, 16 palettes, 4096 tiles. This is the HUD and text plane, and it is always on top.
- Sprites** — up to **381 sprites** on screen, **96 per scanline**. Each sprite is a vertical strip of up to 32 tiles of 16×16 pixels, with per-sprite horizontal and vertical *reduction* and a chain ("sticky") bit that welds a strip to its left neighbour.
- Backdrop** — one solid colour behind everything, from the 65,536-colour pool. That is the entire background layer.

There is no tile-mapped background plane. Every moving and static element — player, enemies, terrain, parallax — is built out of sprites. Sprite count and the per-line limit are what drive your scene

budget.

Two facts catch every newcomer:

- **Sprites can only shrink, never stretch.** The SCB2 scale field is a *reduction* factor. Art must be authored at its largest intended size.
- **Lower sprite slot number is drawn IN FRONT.** Putting a background at slot 1 puts it in front of your hero. Backgrounds belong at high slots.

Colour

Colour is paletted, not direct. 15 colours per character (4 bits of index; index 0 is transparent), 256 palettes of 16 entries in palette RAM at \$400000–\$401FFF, giving 4,096 colours on screen chosen from 65,536.

A Neo Geo colour word is not plain RGB444. Bit 15 is the "dark" bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top 4 bits of R, G and B. The result is **5 bits per channel, 32 levels, in steps of 8** — which is why the artbox quantiser snaps to that lattice rather than to a 4-bit grid.

The **backdrop register is the last word of palette RAM**, at \$401FFE. \$402000 is a mirror that silently does nothing.

Timing

Two interrupts, and that is the whole frame loop.

IRQ	Source	Vect or	Use
1	Vertical blank	\$64	The game tick: read input, run logic, push the frame's sprite list into VRAM
2	LSPC raster timer	\$68	Raster effects: split scrolls, mid-frame palette swaps, status bars

VBlank is roughly 3 ms wide. Everything that writes VRAM from gameplay should be queued and flushed inside it — which is what the engine's render queue does for you.

VRAM

You do not poke VRAM directly. You talk to the LSPC, and it talks to VRAM.

Address	R/W	Register
\$3C0000	W	VRAM address (16-bit word index)
\$3C0000	R	Read data (address register unchanged)
\$3C0002	W	VRAM write data — writing pushes to VRAM
\$3C0004	R/W	Auto-increment, added to the address after every write
\$3C0006	W	Mode register — timer / IRQ / auto-animation speed
\$3C0008	W	Timer high
\$3C000A	W	Timer low
\$3C000C	W	Interrupt clear (bit 1 timer, bit 2 VBL)

All access is word-aligned; long-word and byte access are not allowed. Wait at least 1.4 μ s (16 CPU clocks) after touching the address register before reading. The SDK wraps all of this in named C helpers.

The two CPUs talking

One byte at a time. The 68000 writes a sound code to \$320000; the Z80 reads it on its own port and dispatches to the right music, SFX, FM or SSG handler. The Z80 can post a status byte back through the same address.

You do not write that byte by hand — you call `playMusic()` / `playSFX()` with a named id from `sdk/sound_ids.h`, the SDK encodes it, and the Z80 driver decodes it.

3. What is in the box

```
neogeosdk/
├── sdk/                    68000 headers, runtime, and both 2D engines
│   ├── 2d_engine/         the C engine (ng_*.c / ng_*.h)
│   ├── 2d_engine_plus/    the C++ engine (ng_*.cpp / ng_*.hpp), same ABI
│   ├── ng_fix/ ng_audio/  ng_video/ ng_scene/ ng_show/ bsp/
│   ├── neogeo.h macro.h  hardware registers and inline helpers
│   └── sound_ids.h        named music / SFX / FM / SSG ids
├── games/                 one directory per game, each self-contained
│   ├── demo/              the 25-chapter engine reel (id 777)
│   ├── demo_plus/        C++-engine smoke test (id 778)
│   ├── skylance/         Sky Lance, a complete shooter (id 779)
│   ├── helloworld/       minimal FIX-text + one sample (id 772)
│   ├── tutorial/         minimal engine-loop skeleton (id 555)
│   └── neogegame/        empty starting point for your game(id 775)
├── artbox/                PNG → sprite/FIX/C-ROM pipeline + Artbox Studio
├── sound/                 YM2610 stack: Z80 driver, MML, FM, SSG, samples
├── docs/                  this documentation set
├── tools/                 validators and the manual generator
├── dist/ roms/ hash_eagle/ build output and MAME hash sets
├── Makefile               Linux / WSL build
└── MakefileWin32.mak     native Windows build
```

4. The two engines

The SDK ships the same 2D engine twice: once in plain C (`sdk/2d_engine/`), once in C++14 (`sdk/2d_engine_plus/`). They are not different engines — they are the same design with the same public extern "C" ABI, so a C game can link against the C++ build and vice versa.

Build	Choose it when
<code>sdk/2d_engine</code>	You are writing C. Smallest code-size footprint, simplest call chain. This is the default.
<code>sdk/2d_engine_plus</code>	You are writing C++. You want member methods (<code>cam.follow(...)</code>), singletons (<code>CharManager::instance()</code>), and the scene-stack helpers.

Pick one per ROM with the `USE_2D_PLUS` make flag:

```
make p1          # link the C engine (default)
make p1 USE_2D_PLUS=1  # link the C++ engine
```

The C++ build compiles under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no `malloc`, no floating point, no order-dependent global constructors.

Full call-by-call references:

- `API_2D_ENGINE_C.md`
- `API_2D_ENGINE_CPP.md`

5. Where to go next

If you want to...	Read
Build your first ROM in ten minutes	HELLO_WORLD.md
Understand the whole build, layer by layer	PROGRAMMERS_MANUAL.md
Look up an engine call	API_2D_ENGINE_C.md / API_2D_ENGINE_CPP.md
Look up a bare-metal SDK call	../SDK_API_GUIDE.md
Convert artwork	ARTBOX_PIPELINE.md
Make sound	SOUND_DRIVER.md, SOUND_STUDIO_GUIDE.md
Start a new game	ADDING_A_GAME.md
See what each shipped game demonstrates	GAMES.md, DEMO_CHAPTERS.md
Stay inside the frame budget	performance_rules.md
Look up a helper script	TOOLS.md
Debug on real hardware timing	GDB_GUIDE.md

6. Licence and intended use

MIT. The SDK is intended for lawful homebrew development, technical research, education, preservation-oriented study, and authorised consulting work. It must not be used for unauthorised copying, distribution, reproduction, sale, conversion, or operation of copyrighted commercial software.

PART 2

Programmer's Manual

The build system, the project layout, the game contract, the engine tick, the asset pipelines, and the frame budget.

`docs/PROGRAMMERS_MANUAL.md`

Eagle Software · Neo Geo SDK v1.7.0

The working manual for building Neo Geo software with this SDK: the build system, the project layout, the game contract, the engine tick, the asset pipelines, and the rules that keep a frame inside 16.7 ms.

Read INTRODUCTION.md first if you have not — it covers the hardware this manual assumes you know. For a first ROM, follow HELLO_WORLD.md. For call-by-call detail, use API_2D_ENGINE_C.md, API_2D_ENGINE_CPP.md and ../SDK_API_GUIDE.md.

Part I — The build

1. Two makefiles, one build

Host	Command
Linux, WSL	make ...
Windows (native)	make -f MakefileWin32.mak ...

They implement the same targets and produce byte-identical ROMs. Everything in this manual applies to both; only the invocation differs.

2. Selecting a game

Every game directory carries a game.cfg naming itself, and the repo root carries one naming whichever game you are working on. The build cross-checks GAME= against the cfg and stops if they disagree — which is what stops you from silently building the demo's art into another game's ROM.

```
make GAME=skylance GAME_CFG_FILE=games/skylance/game.cfg all
```

Setting CURRENT_GAME in the root game.cfg lets you drop both arguments for the game you work on most.

3. The five stages

make all runs them in order. Each is also a target on its own.

Stage	Target	Produces
1	art	Sprite C-ROMs (<id>-c1.c1, <id>-c2.c2), screens.c, sprite_meta.h, palettes
2	sfix	The S1 FIX ROM (<id>-s1.s1), 128 KB
3	sound	ADPCM samples, V-ROM, and the M1 Z80 driver ROM
4	p1	The 68000 program ROM (<id>-p1.p1), 512 KB
5	*(in p1)*	hash_eagle/<game>/neogeo.xml — the MAME hash set

In day-to-day work you will run p1 almost exclusively; it takes seconds on a warm tree. Re-run art when you change a PNG, sound when you change audio, and all before you ship.

Full target list

```

make all                art + sfix + sound + p1
make p1                 68000 program ROM only
make aes / mvs          force the AES or MVS platform variant
make art                rebuild sprite C-ROMs from artbox
make art-crt            same, through the CRT-preprocessing pipeline
make sfix / srom        rebuild the S1 FIX ROM
make sound              samples + vrom + fmpatches + fm + mml + ssgconfig + ssg + mlrom
make vrom               rebuild the V-ROM from ADPCM assets
make mlrom              the shipping ASM Z80 driver build
make mlrom-c            the experimental C driver build
make compare-driver     build both and diff the ROM images
make fm / fmpatches / mml / ssg / ssgconfig / samples
make dist               package dist/roms/<game>.zip
make test               run in MAME (MVS, sp-s2.spl BIOS)
make test-aes           run in MAME (AES, unibios22)
make test BIOS=unibios40
make debug              run MAME with the debugger attached
make debug-build        P1 with debug symbols and a linker map
make debug-artifacts    size, symbols, readelf, map and disassembly dumps
make gdb / gdb-trace / gdb-remote
make bios-list          list the BIOS images MAME can see
make games-list         list the games in the tree
make menu / help        the interactive launcher
make clean / sound-clean / art-clean / clean-all

```

4. Build behaviour worth knowing

- ROMs are written per game: roms/demo/777-p1.p1, roms/skylance/779-p1.p1.
- hash_eagle/<game>/neogeo.xml is regenerated on every p1 with correct CRC/SHA1 and loadflag="load16_word_swap" on the P-ROM. MAME will not load the cart without it.
- P1 generation crops to the full 512 KB window (0x080000) *before* the byte swap and padding. This is the required format for both MAME and hardware — a ROM built without the crop boots to garbage.
- make samples uses the bundled Python WAV converter by default. Set SOX=/path/to/sox only if you specifically want the SoX path.
- make dist puts the ROM files at the archive root, no subfolder.
- make mlrom-asm is the authoritative M1 path; the C driver is for comparison.

5. Debugging

```

make debug-build        # P1 with symbols + linker map
make debug-artifacts    # size / nm / readelf / map / disassembly into dump/
make gdb                # GDB on out/game
make gdb-remote GDB_REMOTE=localhost:23946
make gdb-trace          # scripted batch trace into dump/gdb_trace.txt
make debug              # MAME with the debugger attached

```

Symbol addresses come from m68k-unknown-elf-nm -n out/game, which is how you find a variable to watch from a MAME Lua script. MAME's -autoboot_script plus install_write_tap is the practical way to trace a value across frames — keep the tap handle in a global or it is collected and silently stops firing.

Full workflow: GDB_GUIDE.md.

Part II — Writing the game

6. The contract

A Neo Geo cartridge boots into the BIOS, not into `main()`. The BIOS calls back into the cart for power-on, eye-catcher, game and title; `user.c` provides those hooks and you rarely touch it.

What you write is:

```
void game_boot(void);    /* once, after the BIOS hands over */
void game_frame(void);   /* once per frame, forever */
```

`game_frame()` returns every frame, so anything that must persist across frames is `static` or file-scope. A plain `local` is gone before the next call — the most common first bug on this platform.

7. Project layout

```
games/mygame/
├ game.mk          GAME_NAME, GAME_ID, GAME_SCENES, GAME_EXTRA_INCLUDES
├ game.cfg         CURRENT_GAME=mygame
├ main.c           game_boot() / game_frame(), and the generated screen table
├ user.c           BIOS hooks – interrupt vectors, USER dispatch
├ eyecatcher.c     generated by the artbox
├ neogeo.ld        linker script
├ neogeo_mvs.c     MVS platform stub
├ neogeo_aes.c     AES platform stub
├ scenes/          your code, listed in GAME_SCENES
├ artbox/in/       source PNGs
└ sound/           MML, FM, SSG and WAV sources
```

`game.mk`:

```
GAME_NAME = My Game
GAME_ID   = 780
GAME_SCENES = title stage boss
GAME_EXTRA_INCLUDES = -Igames/other/artbox # optional
```

`GAME_ID` becomes the ROM filename prefix and must be unique in the tree. In use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

`GAME_SCENES` is an explicit list, not a wildcard, so helper files that are `#included` elsewhere do not get compiled twice.

Details: `ADDING_A_GAME.md`.

8. Driving the engine

The minimum:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void) { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

`ng_game_engine_frame()` runs a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — then flushes the render queue inside `VBlank`. You attach your game at five points:

```
ng_game_engine_set_hooks(before_logic,
                        collision_logic,
                        after_events,
                        before_draw,
                        after_draw);
```

Use the hooks rather than reimplementing the tick. Ordering inside the tick exists for reasons that are not obvious from outside: physics resolves after character movement but before the camera reads positions, and the render queue flush must be the last thing that happens.

9. Characters

ng_chars is a fixed pool of 64. Add, bind art, set a body, and the engine moves, animates, culls and draws them.

```
NGCharacter *c = chars_add(KIND_PLAYER, x, y);
ng_char_bind_asset(c, &asset_view);
ng_char_set_body(c, 0, 0, 24, 40);
ng_char_set_speed_fp(c, vx_fp, vy_fp);
ng_char_set_priority(c, NG_RENDER_BAND_PLAYER, 0);
```

Two traps that have cost real debugging time:

****ng_chars_count()** is not an array bound. ****** It returns the number of *active* objects. Once a slot frees, the active count is lower than the highest occupied index, and a loop bounded by it silently skips the tail. Iterate NG_MAX_CHARS and skip inactive slots:

```
for (i = 0; i < NG_MAX_CHARS; i++) {
    NGCharacter *c = chars_at(i);
    if (!c || !c->active) continue;
    ...
}
```

****Remove with ng_chars_remove().**** Setting a "destroy pending" life state is not acted on by the engine — the object stays active, the pool fills, and new spawns start failing. This presents as "the player can only fire six shots, then never again".

For collision over two pools, gather index lists first and then test list against list. A naive double loop over NG_MAX_CHARS is 64 × 64 rectangle tests per frame.

10. Sprites, slots, and priority

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction. Earlier revisions of the engine headers claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters.

Slots	Use
1-32	Background layers, parallax — the true back
33-95	Title and one-off generated screens
96-223	Characters and NPCs, assigned by ng_chars
224-255	Effects
256-287	Particles
288-379	Foreground / temporary effects

NG_SPR_BG0_FIRST and NG_SPR_BG1_FIRST predate the correction; the names read as if they were about layer order and they are not. Prefer explicit slot numbers in new code.

Other hardware facts that shape your code:

- **SCB2 only shrinks.** There is no stretch. Author art at its largest intended size and scale down. A "30 %" sprite is 0x4C-ish in the shrink field, not a multiplier you can exceed.
- ****SCB3's Y field is 496 - y, nine bits.**** So y and y + 512 are the same position, and negative Y wraps correctly — you do not need to clamp a sprite moving off the top.
- **381 sprites, 96 per scanline.** The per-line limit is what actually bites: a wide row of enemies at the same Y will drop sprites long before the total budget is reached.
- **A page is 16 × 16 tiles.** A scrolling background must be exactly one page in the scroll axis and seamless at the join, because you cannot stretch it to fit.

11. The FIX layer

40 × 32 cells of 8 × 8 pixels, of which **28 rows are visible**. Visible row y is map row y + 2; the ng_fix_* helpers apply that for you.

The FIX map word is (palette << 12) | tile — **four bits of palette**, so banks 0–15 only. A game with more than 16 FIX palettes has to fold duplicates; sky_fix_palettes_init() in Sky Lance is a worked example of doing that at boot.

The BIOS SFIX font's space glyph is opaque. Printing " " punches a solid colour-index-2 plate through whatever is behind it. To leave a cell genuinely empty use ng_fix_blank_cell(); tile 0xFF is the transparent one.

The layer is always on top of every sprite. That is why it is the right place for a HUD and the wrong place for anything that should pass behind a character.

12. Palettes and the backdrop

Palette RAM is \$400000–\$401FFF. A colour word is not RGB444: bit 15 is the dark bit, bits 14–12 carry the least-significant bit of each channel, and bits 11–0 carry the top four bits of R, G and B — **5 bits per channel, 32 levels, in steps of 8**.

****The backdrop register is the last word of palette RAM, \$401FFE.**** \$402000 is a mirror that silently does nothing; writing there is a frequent cause of "my backdrop colour is ignored".

ng_palette_fx runs fades, flashes, pulses and colour cycles on a bank, all routed through the render queue so an effect started in game logic still lands inside VBlank.

13. Input

```
ng_joystick_update();                /* once per frame, first */
if (ng_joy_pressed() & BUTTON_A) { ... } /* edge */
if (ng_joystick_state()->raw & JOY_LEFT) { ... }
```

poll_joystick() gives you the raw current state; acting on it directly fires once per frame for as long as the button is held. ng_joy_repeat() gives menu-style auto-repeat, ng_joy_held_frames() gives charge timing, and ng_joy_special_qcf() / ng_joy_special_dp() recognise motion inputs.

Part III — Assets

14. Art

Drop PNGs into `games/<game>/artbox/in/<category>/` and run `make art`. The pipeline quantises to the Neo Geo colour lattice, builds palettes, packs tiles into C-ROMs, and generates `screens.c` (one `showScreenN()` per asset) and `sprite_meta.h` (geometry per asset).

Categories are processed in a fixed order — backgrounds, characters, effects, eyecatcher, npc, screens, titles — and any other directory is appended alphabetically. **Asset ids are the 1-based position in that order**, so inserting a file in the middle renumbers everything after it. The shipped games pad filenames to control ordering; do the same.

characters and npcs share one derived 15-colour master palette per group. Other categories get per-asset palettes. That is why a sprite that looks wrong after being moved between categories is usually a palette problem, not a conversion problem.

Full detail, including the quantiser and the Artbox Studio GUI: `ARTBOX_PIPELINE.md`.

15. Sound

`soundInit()` once at boot, then `playMusic()`, `playSFX()`, `playSFXB()`, `playFMTrack()`, `playSSGTrack()` with ids from `sdk/sound_ids.h`.

Every call posts one byte to a second CPU. Nothing is synchronous, and multi-byte commands (a prefix plus a parameter) must be spaced by a `waitVbl()` or the second byte can be dropped.

Full detail: `SOUND_DRIVER.md`. Composing tools: `SOUND_STUDIO_GUIDE.md`.

Part IV — Staying inside the frame

16. The budget

Resource	Limit
CPU	~12 MHz 68000, ~200,000 cycles per frame
VBlank	~16 scanlines, ~1.1 ms — all VRAM writes must fit
Sprites	380 total, 96 per scanline
Palettes	16 entries per bank, 256 banks
FIX	40 × 32 cells, 28 rows visible, 16 palettes

17. The rules

****No malloc during gameplay.**** Every engine subsystem uses a fixed pool. Dynamic allocation fragments RAM and produces non-deterministic pauses.

Pool	Size	Constant
Characters	64	<code>NG_MAX_CHARS</code>
NPCs	32	<code>NG_MAX_NPCs</code>

Pool	Size	Constant
Particles	32	NG_PART_MAX_PARTICLES
Palette FX	8	NG_PALFX_MAX_SLOTS
Render queue	128	NG_RQ_MAX_CMDS

****No float or double.**** There is no FPU; every float costs hundreds of cycles in software emulation. Use 16.16 fixed-point from `ng_fixed.h`.

No division in the frame loop. DIVS/DIVU cost 76–158 cycles each. Use `ng_sin_tab[]`, `ng_cos_tab[]`, `ng_shrink_tab[]`, right shifts, and fixed-point multiply.

Write only what changed. Sprite groups carry dirty flags; the FIX layer has a dirty-cell cache. Rewriting an unchanged sprite chain every frame is the fastest way to blow the VBlank budget.

Queue every VRAM write. Post to the render queue from game logic and let `ng_render_queue_flush()` drain it in VBlank. Writing VRAM outside VBlank produces tearing and corrupt tiles.

Budget sprites per line, not per screen. 96 is the number that matters.

More: `performance_rules.md`.

Appendix A — Register quick reference

Address	R/W	Register
\$3C0000	W	VRAM address (word index)
\$3C0000	R	VRAM read data
\$3C0002	W	VRAM write data
\$3C0004	R/W	Auto-increment
\$3C0006	W	Mode — timer / IRQ / auto-animation
\$3C0008 / \$3C000A	W	Timer high / low
\$3C000C	W	IRQ acknowledge (bit 1 timer, bit 2 VBL)
\$320000	R/W	Sound command port to / from the Z80
\$400000–\$401FFF	R/W	Palette RAM
\$401FFE	W	Backdrop colour
\$3A000F / \$3A001F	W	Palette bank select
\$3A0011	W	Shadow / display dim
\$300001	W	Watchdog kick

Word-aligned access only. Wait $\geq 1.4 \mu\text{s}$ after touching the VRAM address register before reading.

Appendix B — Interrupts

IRQ	Source	Vector	Acknowledge
1	V-Blank	\$64	MOVE.W #4, \$3C000C

IRQ	Source	Vector	Acknowledge
2	LSPC timer	\$68	MOVE.W #2, \$3C000C

Kick the watchdog (MOVE.B D0, \$300001) once per frame in the VBlank handler or the board resets.

Appendix C — Common symptoms

Symptom	Cause
Sprite invisible behind the background	Slot priority — higher is in front
Sprite bigger than intended, cannot shrink further	SCB2 only reduces; re-author the art
Backdrop colour ignored	Wrote \$402000 instead of \$401FFE
Text draws solid blocks	BIOS space glyph is opaque; use <code>ng_fix_blank_cell()</code>
FIX palette wraps onto text colours	Only 4 palette bits — fold duplicates below 16 banks
Pool fills and spawns stop	Objects marked destroyed but never <code>ng_chars_remove()</code> d
Loop skips the last objects	<code>ng_chars_count()</code> used as an array bound
Second byte of a sound command lost	Prefix and parameter not spaced by <code>waitVbl()</code>
Sprites dropped along one row	96-per-scanline limit
Green self-test screen on boot	BIOS self-test, not your ROM — try BIOS=unibios40
Tearing or corrupt tiles	VRAM written outside VBlank instead of queued

PART 3

Tutorial: Hello World

From a clean checkout to a running ROM, then into a game of your own.

`docs/HELLO_WORLD.md`

Eagle Software · Neo Geo SDK v1.7.0

This tutorial takes you from a clean checkout to a running ROM that prints text on screen and plays a sample when the player presses **A**. It uses the `games/helloworld` project that ships with the SDK, then shows you how to copy it into a game of your own.

Budget: about ten minutes, most of it the first art build.

0. Before you start

You need the toolchain installed and `SDKHOME` set. See `DEPENDENCIES.md` and the installation sections of `../README.md`, or run the installer for your platform from `install/`.

Quick check:

```
m68k-unknown-elf-gcc --version      # 68000 cross-compiler
wla-z80 --version                  # Z80 assembler for the sound driver
python3 --version                  # 3.10+, with Pillow and NumPy
make --version
```

1. Build and run it

```
# Linux / WSL
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test

# Windows
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg all
make -f MakefileWin32.mak GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test
```

`all` runs five stages in order — **art**, **sfix**, **sound**, **p1**, and the hash-set generation MAME needs. `test` launches MAME on the result.

Every game in the tree carries its own `game.cfg` naming itself, and the repo-root `game.cfg` names whatever game you are working on. The `GAME_CFG_FILE=` argument is what lets you build a different game without editing the root file. If the two disagree, the build stops and tells you.

You should see `Hello World!` and `Press A for sound` on a black screen. Press **A**: sample 0 plays.

2. What actually ran

A Neo Geo cartridge boots into the BIOS, not into your `main()`. The BIOS calls back into the cartridge for four things — power-on, eye-catcher, game, and title — and every game in this SDK provides those hooks in `user.c`. You almost never edit that file.

What you *do* write is two functions in `main.c`:

```
void game_boot(void);    /* called once, after the BIOS hands over */
void game_frame(void);   /* called once per frame, forever */
```

That is the whole contract for a minimal game.

`games/helloworld/main.c`

```

#include <stdint.h>
#include "sdk/2d_engine/ng_fix.h"
#include "sdk/neogeo.h"
#include "sdk/macro.h"    /* button and joystick bit masks */

void game_boot(void) {
    ng_fix_init();          /* set up the FIX text layer */
    ng_fix_clear();         /* wipe whatever the BIOS left */
    ng_fix_puts(3, 15, "Hello World!", 0); /* x, y in 8x8 cells; 0 = palette */
    ng_fix_puts(3, 17, "Press A for sound", 0);
    soundInit();            /* boot the Z80 + YM2610 */
}

static uint16_t prev_joy = 0; /* file scope: survives frames */

void game_frame(void) {
    uint16_t joy = poll_joystick(); /* JOY_* / BUTTON_* bitfield */

    if (!(prev_joy & BUTTON_A) && (joy & BUTTON_A)) {
        playSFX(0); /* rising edge only */
    }
    prev_joy = joy;
}

```

Three things in that file are worth internalising, because they apply to every Neo Geo program you will write:

The FIX layer is cell-addressed, not pixel-addressed. `ng_fix_puts(3, 15, ...)` means column 3, row 15 of a 40×32 grid of 8×8 tiles. Only 28 rows are visible, and the SDK's FIX helpers already account for the two-row offset between map rows and screen rows.

Input is polled, not queued. `poll_joystick()` returns the *current* state. If you act on the raw value you will fire once per frame for as long as the button is held, which is almost never what you want — hence the `prev_joy` edge test. The 2D engine's `ng_joystick` module does this bookkeeping for you (`ng_joy_pressed()`), and that is what a real game should use.

Sound is a one-byte command to another CPU. `playSFX(0)` writes a sound code to $\$320000$. The Z80 picks it up on its own port and dispatches it. Nothing about that call is synchronous; do not expect a return value or a completion signal.

Why `prev_joy` is at file scope

A static inside `game_frame()` would work too. What must *not* happen is a plain local — `game_frame()` returns every frame, so a stack local is gone before the next call. This is the single most common first bug.

3. Change something

Edit the text, then rebuild only the 68000 side:

```

make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg p1
make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg test

```

`p1` skips art and sound and just recompiles and relinks the program ROM. On a warm tree it takes a couple of seconds, and it is the target you will use ninety percent of the time.

Some things to try:

Change	What you learn
<code>ng_fix_puts(3, 15, "Hello World!", 1)</code>	Palette 1 instead of 0. FIX palettes are 4 bits — banks 0-15 only.

Change	What you learn
Add <code>setBACKDROP(0x8001);</code> in <code>game_boot()</code>	The backdrop register is the whole background layer.
Print joy with <code>ng_fix_put_u16(3, 19, joy, 0, 0)</code>	Watch the input bitfield live.
<code>playSFX(1)</code> on <code>BUTTON_B</code>	Sample ids come from <code>sound/samples/in_wav_a/</code> .

4. Add a sprite

`ng_fix_*` draws text. Anything that moves is a sprite, and sprites come from the artbox.

1. Drop a PNG into `games/helloworld/artbox/in/characters/`.
2. Re-run the art stage: `sh make GAME=helloworld GAME_CFG_FILE=games/helloworld/game.cfg art`
3. The pipeline writes `games/helloworld/artbox/sprite_meta.h` with one entry per asset, plus the C-ROM data the hardware reads tiles from.
4. Include `sprite_meta.h` and bind the asset to a character with `ng_char_bind_asset()`, or draw it directly with the `sprite-group` API.

The asset id is its 1-based index in the generated manifest, and the order is decided by directory (backgrounds, characters, effects, eyecatcher, npc, screens, titles, then any other directory alphabetically) and then by filename. Adding a file in the middle of a category renumbers everything after it, so the shipped games pad names to control ordering. `ARTBOX_PIPELINE.md` covers this properly.

5. Turn it into your own game

`games/neogeogame` (id 775) is the same skeleton with nothing in it, ready to be renamed. To start clean:

```
cp -r games/helloworld games/mygame
```

Then edit **four** things:

File	Change
<code>games/mygame/game.mk</code>	<code>GAME_NAME</code> , and a <code>GAME_ID</code> no other game uses
<code>games/mygame/game.cfg</code>	<code>CURRENT_GAME=mygame</code>
<code>games/mygame/main.c</code>	your <code>game_boot()</code> / <code>game_frame()</code>
<code>games/mygame/artbox/in/</code>	your PNGs

`GAME_ID` becomes the ROM filename prefix (777-p1.p1, 779-c1.c1, ...), so it has to be unique across the tree or two games will overwrite each other's output. Ids in use: 555 tutorial, 772 helloworld, 775 neogeogame, 777 demo, 778 demo_plus, 779 skylance.

If your game needs more than `main.c`, add scene files under `games/mygame/scenes/` and list them in `GAME_SCENES` in `game.mk` — the build compiles exactly that list, so helper files that are `#included` elsewhere do not get compiled twice.

Full instructions: `ADDING_A_GAME.md`.

6. Stepping up to the engine

helloworld talks to the hardware directly. games/tutorial (id 555) is the same size but drives the 2D engine instead:

```
#include "sdk/2d_engine/ng_engine.h"

void game_boot(void) { ng_game_engine_init(); soundInit(); }
void game_frame(void) { waitVbl(); ng_game_engine_frame(); }
```

ng_game_engine_frame() runs the whole engine tick in order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. You hook into it at five points with ng_game_engine_set_hooks(): before logic, collision, after events, before draw, after draw.

From there:

- GAME_ENGINE_LAYER.md — how the engine tick fits together
- API_2D_ENGINE_C.md — every C call, by module
- API_2D_ENGINE_CPP.md — the C++ build
- GAMES.md — what each shipped game demonstrates, and games/skylance as a complete worked example

7. When it does not work

Symptom	Cause
ERROR: GAME mismatch	GAME= and CURRENT_GAME in the cfg disagree. Pass the right GAME_CFG_FILE=.
Green screen with test text on boot	The BIOS self-test, not your ROM. It is normal on some BIOS revisions; make test BIOS=unibios40 skips it.
Blank white or black screen	Usually a backdrop or palette that was never loaded. Check setBACKDROP() and that your palettes are uploaded before you draw.
Text draws as solid blocks	The BIOS SFIX font's space glyph is <i>opaque</i> colour index 2. To leave a cell truly empty use ng_fix_blank_cell(), not " ".
Sprite is invisible	Lower slot = in front. A background parked at a low slot covers everything. Also check the sprite was not left with SCB3 = 0x8000.
Sprite is the wrong size	SCB2 only shrinks. Author art at full size and scale down.
Link error about ng_screen_table	Only if you removed the weak fallback in ng_bg.c; a game with no generated screens links against it.

The demo ROM prints a chapter number in the top-right corner of every scene precisely so problems can be reported by number — see DEMO_CHAPTERS.md.

PART 4

SDK Library Reference

The bare-metal 68000 helpers declared in `sdk/neogeo.h` - sprite control blocks, palettes, VRAM, the FIX layer, timing, input, and sound.

`SDK_API_GUIDE.md`

v1.7.0 quick reference - *Sprite slot priority: **HIGHER slot number = drawn IN FRONT.** This is the observed hardware direction; earlier revisions of this guide and of `ng_sprite_pool.h` claimed the opposite, and the result was backgrounds parked at "behind" slots drawing over characters. Backgrounds belong at slots 1-32, characters at 96-223, foreground effects at 288+. - The backdrop register is the ****last word of palette RAM, \$401FFE****. \$402000 is a mirror that silently does nothing. - The FIX layer is 40 x 32 cells with **28 rows visible**; visible row *y* is map row *y* + 2. The FIX map word is `(pal << 12) | tile`, so there are only **16 palette banks**. - The BIOS SFIX font draws its space glyph as an **opaque** colour-index-2 plate. Use `ng_fix_blank_cell()` to leave a cell genuinely empty. - SCB2 can only **shrink**, never stretch. SCB3's Y field is 496 - *y* in nine bits, so *y* and *y* + 512 are identical and negative Y wraps. - C++14 engine: build with `USE_2D_PLUS=1` (engine in `sdk/2d_engine_plus/`). `games/demo_plus` (id 778) links only against the C++ engine and exists to keep the two builds ABI-compatible. - Makefile accepts `GAME_EXTRA_INCLUDES` (set by a game's `game.mk`) so a game can pull in another game's `artbox/header` path without duplicating data. - Every game carries its own `game.cfg`, so all six build the same way: `make GAME=<name> GAME_CFG_FILE=games/<name>/game.cfg all`.*

This guide covers the public 68000-side SDK helpers declared in `sdk/neogeo.h`.

For installation, SDKHOME layout, WSL usage, and Makefile targets, see:

- `README.md`
- `docs/GAME_ENGINE_LAYER.md`
- `docs/ARTBOX_PIPELINE.md`
- `docs/INTRODUCTION.md`
- `docs/PROGRAMMERS_MANUAL.md`
- `docs/API_2D_ENGINE_C.md`
- `docs/API_2D_ENGINE_CPP.md`
- `docs/MAKEFILE_INTEGRATION.md`

The SDK is organized in five layers:

1. Sprite control block builders
2. Palette and VRAM helpers
3. FIX layer and HUD helpers
4. Timing and input helpers
5. YM2610 sound helpers

Sprite Control Block Helpers

Use these helpers before calling `vram_sprite()` or the sprite move helpers.

Function	Purpose
<code>setSCB2(x_shrink, y_shrink)</code>	Build the sprite shrink word.
<code>setSCB3(y_pos, sticky_flag, height_factor)</code>	Build the sprite Y/sticky/height word.
<code>setSCB4(x_pos)</code>	Build the sprite X position word.
<code>setSCB1_2(pal_offset, tile_offset, bit3_autoanim, bit2_autoanim, vflip, hflip)</code>	Build the per-tile attribute word.
<code>setFIXDATA(palette_index, tile_number)</code>	Build a FIX layer tile word.

Palette and VRAM Helpers

These functions write directly to Neo Geo VRAM and palette RAM.

Function	Purpose
setBACKDROP(backdrop_color)	Set the backdrop color register.
load_palettes(palette, palette_offset)	Copy 16 palette entries into the palette RAM address passed in palette_offset.
vram_init(start, vram_inc)	Set the VRAM address and auto-increment registers.
setpal(...)	Fill a 16-entry palette buffer in RAM.
vram_SCB1(scb1_tiles, scb1_attrs, tiles_number)	Write SCB1 tile/attribute pairs.
vram_sfix(vram_inc, fix_addr, fix_data)	Write a FIX tile with explicit increment and address.
vram_sfix1(fix_data)	Write a FIX tile using the current VRAM cursor.
vram_SCB234(scb_addr, scb_value)	Write a single SCB2/3/4 value.
vram_sprite(...)	Upload a full sprite entry with SCB1/2/3/4 data.
vram_sprite_mv(x, vram_offset, scb4)	Update a sprite X position in VRAM.
vram_sprite_mv(y, vram_offset, scb3)	Update a sprite Y/height word in VRAM.

FIX Layer and HUD Helpers

Use these helpers for text, counters, debug overlays, and BIOS message coordination.

Function	Purpose
fixtext_out(x, y, text, pal)	Write an ASCII string to the FIX layer.
fixtext_out1(x, y, tiles, pal, count)	Write a tile array to the FIX layer.
fixtext_out2(x, y, a, b, c, mod, tiles, pal, count)	Variant helper used for generated FIX strings.
fix_svalue1(x, y, value, pal, offset)	Display a numeric value using a custom digit tile offset.
fix_svalue(x, y, value, pal)	Display a numeric value using the default ASCII digit offset.
display_digit(x, y, value, pal, offset)	Display a uint32_t value without floating-point helpers.
displayCreditP1() / displayCreditP2()	Show credit counters using BIOS memory.
mess_outtest()	Demo helper for the BIOS message output path.
setfix()	Enable the S-FIX border behavior used by the SDK demo.
setBIOSMESSBusy() / setBIOSMESSReady()	Gate access to the BIOS message output path.

Timing and Housekeeping Helpers

Function	Purpose
clearRAM()	Clear the SDK work RAM area.
clearSprs()	Clear the visible sprite list.

Function	Purpose
<code>clearFix()</code>	Clear the FIX layer. Restores BRDFIX (game S ROM) after the BIOS clear call.
<code>waitVbl()</code>	Wait for the next VBlank.
<code>cycle10ms()</code>	Rough 10 ms delay helper.
<code>cycle1s()</code>	Rough 1 second delay helper.
<code>cyclexms1(ticks_10ms)</code>	Delay in coarse 10 ms units.
<code>cyclexs(seconds)</code>	Delay in coarse second units.
<code>cyclexms(millisecond s)</code>	Delay in millisecond-style SDK units.
<code>kickWatchDog()</code>	Service the hardware watchdog register.
<code>sleep1FFF()</code>	Short busy-wait helper used by low-level routines.

Input and Status Helpers

Function	Purpose
<code>poll_joystick()</code>	Read the BIOS-managed joystick state.
<code>read_plcredit()</code>	Read the player 1 credit counter.
<code>read_joy1()</code>	Read the raw joystick hardware ports directly.

Sound API

The full command map, ranges, track indices, and driver behavior live in:

- `sound/SOUND_DRIVER_GUIDE.txt`
- `sound/driver/readme`

High-level sound control categories:

Category	Functions
Core control	<code>soundInit</code> , <code>soundReset</code> , <code>soundStopAll</code> , <code>soundStopMusic</code> , <code>soundCancelFade</code> , <code>soundSceneReset</code>
Raw command path	<code>playSoundtest</code> , <code>soundCommand</code> , <code>isZ80Ready</code>
Music and samples	<code>playMusic</code> , <code>playSFX</code> , <code>playSFXB</code>
FM	<code>playFMDebug</code> , <code>playFMTrack</code> , <code>soundSetFMVolume</code>
SSG	<code>playSSGTrack</code> , <code>soundSetSSGPreset</code> , <code>playInsertCoinSSG</code>
Voice cues	<code>playVoiceCue</code> , <code>playGetReadyVoice</code> , <code>playAttackVoice</code> , <code>playCoinThenReady</code>
Fade / mix	<code>soundFadeOut</code> , <code>soundFadeIn</code> , <code>soundFadeOutSpeed</code> , <code>soundFadeInSpeed</code> , <code>soundSetTempo</code> , <code>soundSetADPCMAVolume</code> , <code>soundSetADPCMBVolume</code> , <code>soundSetSSGVolume</code> , <code>soundApplyMix</code>

Category	Functions
Scene helpers	soundPlayDemoFM, soundPlayTitleMusic, soundPlayGameLoop

Current shipped helper behavior:

- `soundPlayTitleMusic(track)` resets the scene, plays the title gong, then starts the ADPCM-B title theme
- `soundPlayGameLoop(track)` resets the scene and maps the requested loop to a direct ADPCM-B stage or ending TRACK
- `playMusic(track)` remains the explicit path for MML/SSG playback when you want the music engine directly

2D Game Engine Layer

The repository also includes a reusable game engine layer under `sdk/2d_engine/ng_*`.

Primary entry header:

```
#include "sdk/2d_engine/ng_game_engine.h"
```

Core startup:

```
game_engine_init();
```

Per-frame entry:

```
waitVbl();
game_engine_frame();
```

Main runtime modules:

Module	Purpose
<code>ng_actions</code>	const action scripts with commands like FRAME, WAIT, MOVE, SFX, MUSIC, EVENT, LOOP
<code>ng_chars</code>	fixed-size character pool with fixed-point motion and sprite binding
<code>ng_game_events</code>	small queued event system
<code>ng_timers</code>	frame timers
<code>ng_progress</code>	progress counters
<code>ng_status</code>	boolean status flags
<code>ng_properties</code>	live shared values grouped by system
<code>ng_level</code>	level state, world bounds, camera follow, and joystick camera scroll
<code>ng_fix</code>	cached FIX-layer text and numeric output
<code>ng_npcs</code>	fixed-size NPC slots bound to characters and think callbacks
<code>ng_physics</code>	fixed-point gravity, drag, world bounds, and solid-rectangle resolution
<code>ng_border_constraints</code>	invisible trigger rectangles that emit events
<code>ng_sprite_group</code>	grouped Neo Geo sprite upload helpers

Important current build note:

- the linker scripts reserve `game_engine_bss` for `out/ng_*.o`
- `Makefile` and `MakefileWin32.mak` compile and link the `sdk/2d_engine/ng_*` modules by default
- use `docs/MAKEFILE_INTEGRATION.md` to inspect the exact build and linker wiring

Camera helpers:

```
level_set_world_bounds(0, 0, 639, 447);
level_camera_follow(player->x, player->y, 320, 224);
level_camera_joystick(2, NG_CAMERA_AXIS_X, 320, 224); /* horizontal scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_Y, 320, 224); /* vertical scroll */
level_camera_joystick(2, NG_CAMERA_AXIS_BOTH, 320, 224); /* free camera */
```

Characters keep world-space x/y values for physics and collision. The engine subtracts the current level scroll when drawing sprites.

Action-state pattern:

```
static const NGAActionCmd player_run[] = {
    SPEED(0, 0),
    FRAME(tile_run_0, 3),
    FRAME(tile_run_1, 3),
    LOOP()
};

actions_register(ACT_PLAYER_RUN, player_run);
char_action(player, ACT_PLAYER_RUN);
```

Use a per-character callback to map joystick combinations to actions. The demo uses the priority attack, jump, run, idle so combinations produce deterministic results.

NPC and opponent pattern:

```
NGNpc *opponent = npc_spawn(OPPONENT_KIND, CHAR_OPPONENT, 240, 180);
npc_set_think(opponent, opponent_think, 1);

if (ng_rect_hit(char_body_rect(player), char_body_rect(npc_char(opponent)))) {
    char_damage(player, 1);
}
```

Artbox asset categories:

Category	Typical PNG names	Use
background	<code>background_*.png</code> , <code>zz_npc_forest_alley.png</code> , legacy numbered pages	full-screen scene/background pages
main_character	<code>sprite_*.png</code>	player/main-character animation frames
opponent	<code>opponent_*.png</code>	enemy/opponent sprite frames
npc	<code>z_npc_*.png</code>	non-player character sprite frames

The category is written into `artbox/assets_manifest.json` and generated into `artbox/sprite_meta.h` as `NG_ASSET_CATEGORY_*`.

See `docs/ARTBOX_PIPELINE.md` for the full rule and category reference.

Typical Call Patterns

One-time startup

```
soundInit();
clearSprs();
clearFix();
```

Draw a sprite and move it

```
uint16_t scb2 = setSCB2(0x0F, 0xAF);
uint16_t scb3 = setSCB3(496 - y, 1, 9);
uint16_t scb4 = setSCB4(x);

vram_sprite(vram_start, 1, vram_offset, tiles, attrs, tile_count, scb2, scb3, scb4);
vram_sprite_mv(x, vram_offset, setSCB4(x + 8));
```

FIX text and number output

```
fixtext_out(10, 10, "INSERT COIN", 0);
display_digit(20, 15, score, 0, 48);
```

Direct title or stage theme

```
soundPlayTitleMusic(0);
soundPlayGameLoop(SOUND_MUSIC_A);
```

Manual layered sound scene

```
soundSceneReset();
soundSetADPCMAVolume(0x34);
soundSetADPCMBVolume(0xB8);
soundSetSSGVolume(0x08);
soundSetFMVolume(0x0C);

playSFX(SOUND_SFX_1);
playMusic(SOUND_MUSIC_A);
playFMTrack(SOUND_FM_B);
```

Notes

- sdk/neogeo.h is the intended public call surface for the 68000 side.
- The playable sound driver is the assembler implementation in sound/driver/driver.asm.
- make m1rom / make m1rom-asm builds the authoritative ASM sound runtime.
- make m1rom-c builds the experimental C-linked sound runtime for comparison only.
- The current 68000 build no longer links the legacy softfloat/ runtime by default.
- clearFix() calls the BIOS SYS_FIX_CLEAR routine which resets BRDFIX to 0. The SDK now restores BRDFIX (BSET.B #0, REG_BRDFIX) immediately after that call so the game S ROM is always selected. Never call clearFix() without following up with text draws — the game S ROM is restored automatically.
- soundCommand() no longer calls isZ80Ready() after writing the command byte. The trailing poll created a race condition with the Z80 NMI handler that caused a permanent 68k deadlock. Callers

that need to verify Z80 readiness before sending a second command should call `isZ80Ready()` explicitly between commands.

PART 5

2D Engine Reference — C

Every public call in the plain-C 2D engine, grouped by module.

`docs/API_2D_ENGINE_C.md`

****Eagle Software · Neo Geo SDK v1.7.0 · sdk/2d_engine/****

Every public call in the plain-C 2D engine, grouped by module. This file is generated from the headers by tools/gen_api_reference.py — the prototypes here are the prototypes that exist.

The C++ build of the same engine is documented in API_2D_ENGINE_CPP.md; it exposes these same extern "C" entry points plus method wrappers, so the two are link-compatible.

```
#include "sdk/2d_engine/ng_engine.h"  /* everything */
```

Every function carries the NEOGEO_USER linkage tag, which places it in the cartridge's user code section. It is omitted from the signatures below for readability.

Contents

Core — ng_defs · ng_engine · ng_game_events · ng_game_interrupt · ng_game_time · ng_progress · ng_properties · ng_runtime · ng_scene · ng_status · ng_timers

Math — ng_fixed

Video — ng_art_asset · ng_bg · ng_depthfx · ng_fix · ng_palette_assets · ng_palette_fx · ng_render_queue · ng_sprite_group · ng_sprite_pool · ng_sprite_window · ng_vram

Characters — ng_actions · ng_border_constraints · ng_chars · ng_joystick · ng_npcs · ng_physics

World — ng_camera · ng_level

Effects — ng_debug · ng_demo_advanced · ng_feedback · ng_particles

Core

ng_defs

sdk/2d_engine/ng_defs.h — Sizes, engine constants, and the NEOGEO_USER linkage tag.

```
uint16_t ng_abs16(int16_t v);
```

Constant	Value
NG_TRUE	1
NG_FALSE	0
NG_FRAME_RATE	60
NG_FRAMES_PER_SEC	NG_FRAME_RATE
NG_SCREEN_WIDTH	320
NG_SCREEN_HEIGHT	224
NG_SPRITE_CULL_LEFT	-64
NG_SPRITE_CULL_RIGHT	384
NG_SPRITE_CULL_TOP	-128
NG_SPRITE_CULL_BOTTOM	640

Constant	Value
NG_RENDER_BAND_BACK	0
NG_RENDER_BAND_NPC	1
NG_RENDER_BAND_ENEMY	2
NG_RENDER_BAND_PLAYER	3
NG_RENDER_BAND_FX	4
NG_RENDER_BAND_FRONT	5
NG_FP_SHIFT	8
NG_FP_ONE	(1 << NG_FP_SHIFT)
NG_MAX_CHARS	64
NG_MAX_CHAR_KINDS	128
NG_MAX_ACTIONS	256
NG_MAX_GAME_EVENTS	32
NG_MAX_TIMERS	64
NG_MAX_PROGRESS	64
NG_MAX_STATUS	256
NG_PROP_GROUPS	16
NG_PROP_COUNT	32
NG_MAX_BORDER_CONSTRAINTS	128
NG_MAX_NPCS	32
NG_MAX_SOLIDS	64
NG_FIX_WIDTH	40
NG_FIX_HEIGHT	32
NG_SPRITE_MAX_STRIPS	32
NG_SPRITE_MAX_HEIGHT_TILES	32
NG_SPRITE_FULL_XSCALE	0xff
NG_SPRITE_FULL_YSCALE	0xff
NG_PAL_FIX_BASE	0x00
NG_PAL_UI_BASE	0x04
NG_PAL_STAGE_BASE	0x10
NG_PAL_PLAYER_BASE	0x20
NG_PAL_ENEMY_BASE	0x30
NG_PAL_FX_BASE	0x40
NG_PAL_BOSS_BASE	0x50
NG_PAL_CUTSCENE_BASE	0x60

ng_engine

sdk/2d_engine/ng_engine.h — Umbrella include. One #include pulls in every public module.

Types and declarations only — see the header.

ng_game_events

sdk/2d_engine/ng_game_events.h — A small event queue with an optional handler callback.

```
void ng_game_events_set_handler(NGGameEventHandler handler);
uint8_t ng_game_events_send(uint16_t id, uint16_t a, uint16_t b, uint16_t c);
uint8_t ng_game_events_read(NGGameEvent *out);
void ng_game_events_update(void);
uint8_t ng_game_events_count(void);
```

ng_game_interrupt

sdk/2d_engine/ng_game_interrupt.h — The frame tick and the five hooks you attach your game to.

ng_game_engine_frame() runs the whole engine in a fixed order — timers, characters, actions, physics, NPCs, camera, particles, palette effects, events — and flushes the render queue during VBlank. Register your own code at the five hook points rather than reimplementing the tick.

```
void ng_game_runtime_init(void);
void ng_game_engine_set_hooks( NGInterruptHook before_logic, NGInterruptHook collision_logic, NGInterruptHook after_events, NGInterruptHook before_draw, NGInterruptHook after_draw );
void ng_game_interrupt_set_hooks( NGInterruptHook before_logic, NGInterruptHook collision_logic, NGInterruptHook after_events, NGInterruptHook before_draw, NGInterruptHook after_draw );
void ng_game_engine_frame(void);
void ng_game_interrupt(void);
```

ng_game_time

sdk/2d_engine/ng_game_time.h — Frame and stage counters at 60 Hz.

```
void ng_game_time_tick(void);
void ng_game_time_reset_stage(void);
uint32_t ng_game_time_frame(void);
uint16_t ng_game_time_second(void);
uint32_t ng_game_time_stage_frame(void);
```

ng_progress

sdk/2d_engine/ng_progress.h — Bounded 0..max counters — charge meters, load bars, HP ramps.

```
void ng_progress_start(uint8_t id, uint16_t max);
void ng_progress_set(uint8_t id, uint16_t value);
void ng_progress_add(uint8_t id, uint16_t amount);
uint16_t ng_progress_value(uint8_t id);
uint8_t ng_progress_percent(uint8_t id);
uint8_t ng_progress_done(uint8_t id);
void ng_progress_update(void);
```

ng_properties

sdk/2d_engine/ng_properties.h — A grouped int32 property matrix for game-defined values.

```
void ng_prop_set(uint8_t group, uint8_t id, int32_t value);
int32_t ng_prop_get(uint8_t group, uint8_t id);
void ng_prop_add(uint8_t group, uint8_t id, int32_t amount);
void ng_prop_clear_group(uint8_t group);
```

ng_runtime

sdk/2d_engine/ng_runtime.h — Thin alias header — includes the whole engine.

Types and declarations only — see the header.

ng_scene

sdk/2d_engine/ng_scene.h — Scene entry/teardown helpers.

ng_scene_begin() is the "start from a known state" call: it clears hardware and engine state so nothing leaks in from the previous scene.

```
void ng_scene_clean_default(void);
void ng_scene_begin(uint8_t flags, uint8_t wait_vblank);
```

Constant	Value
NG_SCENE_CLEAN_FIX	0x01u
NG_SCENE_CLEAN_SPRITES	0x02u
NG_SCENE_CLEAN_CHARS	0x04u
NG_SCENE_CLEAN_PHYSICS	0x08u
NG_SCENE_CLEAN_EVENTS	0x10u
NG_SCENE_CLEAN_TIMERS	0x20u
NG_SCENE_CLEAN_PROGRESS	0x40u
NG_SCENE_CLEAN_PALETTEFX	0x80u
NG_SCENE_CLEAN_DEFAULT	(\

ng_status

sdk/2d_engine/ng_status.h — A bitset of "has this happened yet" flags.

```
void ng_status_set(uint16_t status_id);
void ng_status_clear(uint16_t status_id);
uint8_t ng_status_has(uint16_t status_id);
void ng_status_toggle(uint16_t status_id);
```

ng_timers

sdk/2d_engine/ng_timers.h — One-shot countdown timers, addressed by id.

```
void ng_timer_start(uint8_t id, uint16_t frames);
void ng_timer_stop(uint8_t id);
void ng_timers_update(void);
uint8_t ng_timer_active(uint8_t id);
uint8_t ng_timer_done(uint8_t id);
uint16_t ng_timer_value(uint8_t id);
uint8_t ng_timer_percent_left(uint8_t id);
```

Math

ng_fixed

sdk/2d_engine/ng_fixed.h — 16.16 fixed-point helpers, sin/cos and shrink lookup tables.

There is no floating point on this target. Everything that would have been a float is 16.16 fixed-point, and every trig or scale lookup is a table.

Types and declarations only — see the header.

Video

ng_art_asset

sdk/2d_engine/ng_art_asset.h — Type queries on generated artbox asset records.

```
uint8_t ng_art_asset_is_type(const NGArtAsset *asset, uint8_t type);
```

Constant	Value
NG_ART_TYPE_SCREEN	0u
NG_ART_TYPE_SPRITE	1u
NG_ART_TYPE_BACKGROUND	2u
NG_ART_TYPE_FIX	3u
NG_ART_TYPE_EFFECT	4u

ng_bg

sdk/2d_engine/ng_bg.h — Two scrolling sprite background layers with parallax factors.

```
void ng_bg_hide_all(void);
void ng_bg_mark_dirty(uint8_t layer);
```

Constant	Value
NG_BG_LAYERS	2

Constant	Value
NG_BG_LAYER0	0
NG_BG_LAYER1	1
NG_SCREEN_TABLE_MAX	200

ng_depthfx

sdk/2d_engine/ng_depthfx.h — Perspective projection, Z-to-shrink, fog palettes, starfields.

The Neo Geo can only shrink sprites, so depth is expressed as reduction plus a palette swap for fog banding.

Constant	Value
NG_DEPTH_CX	160
NG_DEPTH_CY	112
NG_DEPTH_PAL_NEAR	0
NG_DEPTH_PAL_MID	1
NG_DEPTH_PAL_FAR	2
NG_DEPTH_PAL_VERY_FAR	3

ng_fix

sdk/2d_engine/ng_fix.h — The FIX text layer, with a dirty-cell cache.

The layer is 40 x 32 cells of which 28 rows are visible; these helpers apply the row offset for you. Use `ng_fix_blank_cell()` to empty a cell — the BIOS font draws its space glyph as an opaque colour-2 plate.

```
void ng_fix_invalidate_all(void);
void ng_fix_clear(void);
void ng_fix_clear_rect(uint8_t x, uint8_t y, uint8_t w, uint8_t h, uint8_t pal);
void ng_fix_putc(uint8_t x, uint8_t y, char ch, uint8_t pal);
void ng_fix_puts(uint8_t x, uint8_t y, const char *text, uint8_t pal);
void ng_fix_put_u16(uint8_t x, uint8_t y, uint16_t value, uint8_t pal, uint16_t tile_offset);
void ng_fix_put_u32(uint8_t x, uint8_t y, uint32_t value, uint8_t pal, uint16_t tile_offset);
```

Constant	Value
NG_FIX_SAFE_X	1
NG_FIX_SAFE_Y	2
NG_FIX_SAFE_WIDTH	38
NG_FIX_SAFE_HEIGHT	28

ng_palette_assets

sdk/2d_engine/ng_palette_assets.h — Generated palette asset table lookup.

Types and declarations only — see the header.

ng_palette_fx

sdk/2d_engine/ng_palette_fx.h — Fade, flash, pulse and colour cycle on a palette bank.

Every effect is queue-safe: the palette upload goes through the render queue, so an effect started in game logic still lands inside VBlank.

```
void ng_palette_fx_update(void);
void ng_palette_load_bank(uint8_t palette_slot, const uint16_t *pal);
uint8_t ng_palette_load_asset(const NGPaletteAsset *assets, uint16_t count, uint16_t asset_id);
```

Constant	Value
NG_PALFX_MAX_SLOTS	8
NG_PALFX_NONE	0
NG_PALFX_FADE_IN	1
NG_PALFX_FADE_OUT	2
NG_PALFX_FLASH_WHITE	3
NG_PALFX_FLASH_RED	4
NG_PALFX_FLASH_BLUE	5
NG_PALFX_PULSE	6
NG_PALFX_CYCLE	7

ng_render_queue

sdk/2d_engine/ng_render_queue.h — A 128-slot deferred write queue, flushed inside VBlank.

The engine never writes VRAM from the game logic path. Modules post commands here and `ng_render_queue_flush()` drains them during the ~3 ms VBlank window. `ng_rq_free()` tells you how much room is left.

Constant	Value
NG_RQ_MAX_CMDS	128
NG_RQ_MAX_PAL_UPLOADS	8
NG_RQ_SPRITE_POS	0
NG_RQ_SPRITE_TILE	1
NG_RQ_SPRITE_PAL	2
NG_RQ_SPRITE_SHRINK	3
NG_RQ_SPRITE_HIDE	4
NG_RQ_SCB234	5

ng_sprite_group

sdk/2d_engine/ng_sprite_group.h — Dirty-flag sprite chains — write only what changed.

A group is a strip-0 anchor plus N chained strips welded together by the SCB3 sticky bit. Setting a property marks a dirty flag; the flush writes only the words that actually changed.

```

void ng_sprite_group_set_tile_base(NGSpriteGroup *g, uint16_t tileBase);
void ng_sprite_group_set_tile_stride(NGSpriteGroup *g, uint16_t tileStride);
void ng_sprite_group_set_palette(NGSpriteGroup *g, uint8_t palette);
void ng_sprite_group_set_active_rows(NGSpriteGroup *g, uint8_t activeRows);
void ng_sprite_group_set_pos(NGSpriteGroup *g, int16_t x, int16_t y);
void ng_sprite_group_move(NGSpriteGroup *g, int16_t dx, int16_t dy);
void ng_sprite_group_set_scale(NGSpriteGroup *g, uint8_t xScale, uint8_t yScale);
void ng_sprite_group_set_flip(NGSpriteGroup *g, uint8_t hflip, uint8_t vflip);
void ng_sprite_group_set_auto_anim(NGSpriteGroup *g, uint8_t autoAnim4, uint8_t autoAnim8);
void ng_sprite_group_set_visible(NGSpriteGroup *g, uint8_t visible);
void ng_sprite_group_upload(NGSpriteGroup *g);
void ng_sprite_group_update_transform(NGSpriteGroup *g);
void ng_sprite_group_hide(NGSpriteGroup *g);
void ng_engine_init_hardware(uint16_t transparentTile);
void ng_sprite_hide_vram_base(uint16_t spriteBase, uint16_t count);
void ng_sprite_hide_all(void);
void ng_sprite_disable_hw_range(uint16_t first, uint16_t count);
void ng_sprite_park_off(uint16_t spr);
void ng_sprite_park_off_range(uint16_t first, uint16_t count);

```

Constant	Value
NG_SGF_DIRTY_POS	0x01
NG_SGF_DIRTY_TILE	0x02
NG_SGF_DIRTY_PALETTE	0x04
NG_SGF_DIRTY_SHRINK	0x08
NG_SGF_DIRTY_VIS	0x10
NG_SGF_DIRTY_ALL	0x1F
NG_SPRITE_DISABLED_YREG	256u
NG_SPRITE_DISABLED_X	496u
NG_SPRITE_DISABLED_SCB3	((uint16_t)(NG_SPRITE_DISABLED_YREG << 7))
NG_SPRITE_BLANK_TILE	0xFFFFu
NG_SPRITE_BLANK_ATTR	0x0000u

ng_sprite_pool

sdk/2d_engine/ng_sprite_pool.h — The hardware slot map. Constants only, no functions.

HIGHER slot number is drawn IN FRONT. This is the observed hardware direction and it is the opposite of what earlier revisions of this header claimed. Backgrounds belong at low slots, foreground effects at high ones. The legacy names NG_SPR_BG0_FIRST / NG_SPR_BG1_FIRST predate the correction and are kept only for existing code.

Constant	Value
NG_SPR_TOTAL	380
NG_SPR_LAST	379
NG_SPR_TITLE_FIRST	33
NG_SPR_TITLE_LAST	95
NG_SPR_CHAR_FIRST	96
NG_SPR_CHAR_LAST	223

Constant	Value
NG_SPR_FX_FIRST	224
NG_SPR_FX_LAST	255
NG_SPR_PART_FIRST	256
NG_SPR_PART_LAST	287
NG_SPR_TEMP_FIRST	288
NG_SPR_TEMP_LAST	379
NG_SPR_BG0_FIRST	1
NG_SPR_BG1_FIRST	17
NG_SPR_BG_STRIPS	16

ng_sprite_window

sdk/2d_engine/ng_sprite_window.h — Tracks the strips a variable-size sprite currently occupies.

When a sprite shrinks between frames the strips it no longer uses have to be actively hidden, or the old art stays on screen. That bookkeeping is what a window does.

```
void ng_sprite_window_reset(NGSpriteWindow *window);
void ng_sprite_window_set_current(NGSpriteWindow *window, uint8_t current_strips);
void ng_sprite_window_set_shape(NGSpriteWindow *window, uint8_t current_strips, uint8_t current_rows);
void ng_sprite_window_clear(NGSpriteWindow *window);
void ng_sprite_window_clear_tail(NGSpriteWindow *window);
void ng_sprite_window_hide(NGSpriteWindow *window);
```

ng_vram

sdk/2d_engine/ng_vram.h — Direct VRAM sprite-slot clearing helpers.

```
uint16_t ng_vram_scb1_to_sprite_slot(uint16_t scb1_base);
void ng_vram_clear_sprite_slot(uint16_t sprite_slot);
void ng_vram_clear_sprite_range(uint16_t first_sprite, uint16_t count);
void ng_vram_clear_sprite_vram_base(uint16_t sprite_base, uint16_t count);
void ng_vram_clear_all_sprites(void);
```

Characters

ng_actions

sdk/2d_engine/ng_actions.h — Table-driven action scripts with sound and FX hooks.

```
void ng_actions_register(uint16_t action_id, const NGAActionCmd *script);
void ng_actions_set_sound_hooks(NGActionSfxHook sfx_hook, NGAActionMusicHook music_hook);
void ng_actions_set_fx_hook(NGActionFxHook fx_hook);
void ng_char_action(struct NGCharacter *c, uint16_t action_id);
void ng_actions_update(struct NGCharacter *c);
```

ng_border_constraints

sdk/2d_engine/ng_border_constraints.h — Keep characters inside declared screen regions.

```
void ng_border_constraints_load(NGBorderConstraint *list, uint16_t count);
void ng_border_constraints_update(void);
void ng_border_constraints_reset_used(void);
```

ng_chars

sdk/2d_engine/ng_chars.h — The character pool: 64 objects, movement, animation, hitboxes, damage.

ng_chars_draw() hands out hardware strips from NG_SPR_CHAR_FIRST upward. Two things catch people: ng_chars_count() returns the number of ACTIVE objects, not a high-water index, so it is not a safe array bound — iterate NG_MAX_CHARS and skip inactive slots. And a character is removed with ng_chars_remove(); setting a "destroy pending" life state is not acted on by the engine.

```
NGCharacter* chars_add(uint8_t kind, int16_t x, int16_t y);
void ng_chars_remove(NGCharacter *c);
void ng_chars_clear_kind(uint8_t kind);
NGCharacter* chars_find(uint8_t kind);
NGCharacter* chars_at(uint8_t index);
uint8_t ng_chars_count(void);
uint8_t ng_chars_index(NGCharacter *c);
void ng_chars_set_game_interrupt(uint8_t kind, NGCharInterrupt fn);
void ng_chars_reset_slot(uint8_t index);
void ng_chars_begin_scene_arena(uint8_t arena_id);
void ng_chars_clear_arena(uint8_t arena_id);
void ng_chars_set_default_arena(uint8_t arena_id);
void ng_chars_set_fixed_step(uint8_t updates_per_frame);
void ng_chars_update_fixed(void);
void ng_chars_defrag_slots(void);
void ng_chars_update(void);
void ng_chars_draw(void);
void ng_char_set_asset_bounds(NGCharacter *c, uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_char_bind_asset(NGCharacter *c, const NGSpriteAssetView *asset);
void ng_char_set_cull_margin(NGCharacter *c, int16_t l, int16_t r, int16_t t, int16_t b);
void ng_char_set_anim_clip(NGCharacter *c, const NGAAnimClip *clip);
void ng_char_anim_update(NGCharacter *c);
uint8_t ng_char_validate_asset_window(uint16_t tileBase, uint8_t strips, uint8_t rows, uint16_t stride,
    uint16_t tileStart, uint16_t tileEnd);
uint8_t ng_palette_claim(uint8_t palette_slot, uint8_t owner_kind);
void ng_palette_release(uint8_t palette_slot, uint8_t owner_kind);
void ng_char_set_tile_stride(NGCharacter *c, uint16_t stride);
void ng_char_set_body(NGCharacter *c, int16_t x, int16_t y, int16_t w, int16_t h);
void ng_char_set_pos(NGCharacter *c, int16_t x, int16_t y);
void ng_char_set_speed(NGCharacter *c, int16_t vx_px, int16_t vy_px);
void ng_char_set_speed_fp(NGCharacter *c, int32_t vx_fp, int32_t vy_fp);
void ng_char_add_speed_fp(NGCharacter *c, int32_t ax_fp, int32_t ay_fp);
void ng_char_set_priority(NGCharacter *c, uint8_t priority_band, int16_t depth_offset);
void ng_char_damage(NGCharacter *c, uint8_t amount);
void ng_char_heal(NGCharacter *c, uint8_t amount);
NGRect ng_char_body_rect(NGCharacter *c);
NGRect ng_char_hit_rect(NGCharacter *c);
```

Constant	Value
NG_CHAR_LIFE_FREE	0u
NG_CHAR_LIFE_ALLOCATED	1u
NG_CHAR_LIFE_VISIBLE	2u

Constant	Value
NG_CHAR_LIFE_HIDDEN	3u
NG_CHAR_LIFE_DESTROY_PENDING	4u
NG_STRICT_RENDER_VALIDATE	1

ng_joystick

sdk/2d_engine/ng_joystick.h — Edge-detected input, repeat, motion specials, and character control.

Poll once per frame with `ng_joystick_update()`, then read edges with `ng_joy_pressed()`. Acting on the raw held state instead fires every frame the button is down.

```
void ng_joystick_update(void);
const NGJoystickState *ng_joystick_state(void);
uint16_t ng_joy_pressed(void);
uint16_t ng_joy_released(void);
uint16_t ng_joy_repeat(void);
uint8_t ng_joy_direction(void);
uint8_t ng_joy_held_frames(uint16_t mask);
uint8_t ng_joy_special_dp(uint8_t facing_left, uint16_t attack_button_mask);
void ng_joy_apply_hitbox(NGCharacter *c, uint8_t active_frames, int16_t x, int16_t y, int16_t w, int16_t h);
void ng_joy_set_event_handler(NGJoyEventHandler fn);
uint8_t ng_joy_pop_command(NGJoyCommand *out);
```

Constant	Value
NG_EVENT_JOY_JUMP	0x5001
NG_EVENT_JOY_FIRE_LIGHT	0x5002
NG_EVENT_JOY_FIRE_HEAVY	0x5003
NG_EVENT_JOY_HIT	0x5004
NG_EVENT_JOY_SPECIAL_QCF	0x5005
NG_EVENT_JOY_SPECIAL_DP	0x5006
NG_EVENT_JOY_COMMAND	0x5007

ng_npcs

sdk/2d_engine/ng_npcs.h — NPC wrappers: patrol bounds, home position, think callbacks.

```
void ng_npc_remove(NGNpc *npc);
uint8_t ng_npc_count(void);
void ng_npc_set_think(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void ng_npc_set_home(NGNpc *npc, int16_t x, int16_t y);
void ng_npc_set_patrol_bounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_y);
void ng_npcs_update(void);
void ng_npc_think_hover(NGNpc *npc, NGCharacter *c);
```

Constant	Value
NG_NPC_FLAG_PATROL_X	0x0001
NG_NPC_FLAG_PATROL_Y	0x0002
NG_NPC_FLAG_FACE_MOTION	0x0004

ng_physics

sdk/2d_engine/ng_physics.h — Gravity, drag, solid rectangles, and grounded tests.

```
void ng_physics_attach(NGCharacter *c, uint16_t flags);
void ng_physics_detach(NGCharacter *c);
void ng_physics_set_gravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void ng_physics_set_drag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void ng_physics_clear_solids(void);
uint8_t ng_physics_add_solid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void ng_physics_update_pre(void);
void ng_physics_resolve(void);
uint8_t ng_physics_is_grounded(NGCharacter *c);
```

Constant	Value
NG_PHYSICS_GRAVITY	0x0001
NG_PHYSICS_WORLD	0x0002
NG_PHYSICS_SOLIDS	0x0004
NG_PHYSICS_DRAG_X	0x0008
NG_PHYSICS_DRAG_Y	0x0010

World

ng_camera

sdk/2d_engine/ng_camera.h — Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

Constant	Value
NG_SCREEN_W	320
NG_CAM_FREE	0x00
NG_CAM_FOLLOW	0x01
NG_CAM_CINEMATIC	0x02

ng_level

sdk/2d_engine/ng_level.h — Level state, world bounds, scroll and camera helpers.

```

void ng_level_set_id(uint8_t level_id);
void ng_level_set_mode(uint8_t mode);
void ng_level_set_background(uint16_t screen_id);
void ng_level_set_overlay(uint16_t screen_id);
void ng_level_set_backdrop(uint16_t color);
void ng_level_set_fix_palette(uint8_t palette);
void ng_level_set_scroll(int16_t x, int16_t y);
void ng_level_move_scroll(int16_t dx, int16_t dy);
void ng_level_set_world_bounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void ng_level_set_camera(int16_t x, int16_t y, int16_t screen_w, int16_t screen_h);
void ng_level_move_camera(int16_t dx, int16_t dy, int16_t screen_w, int16_t screen_h);
void ng_level_camera_follow(int16_t target_x, int16_t target_y, int16_t screen_w, int16_t screen_h);
void ng_level_camera_input(uint16_t joystick, int16_t speed, uint8_t axes, int16_t screen_w, int16_t screen_h);
void ng_level_camera_joystick(int16_t speed, uint8_t axes, int16_t screen_w, int16_t screen_h);
void ng_level_update(void);

```

Constant	Value	
NG_CAMERA_AXIS_X	0x01	
NG_CAMERA_AXIS_Y	0x02	
NG_CAMERA_AXIS_BOTH	`(NG_CAMERA_AXIS_X	NG_CAMERA_AXIS_Y)`

Effects

ng_debug

sdk/2d_engine/ng_debug.h — A FIX-layer performance overlay.

Compile with -DNG_DEBUG_PERF=1 to enable.

```

void ng_debug_draw(void);
void ng_debug_clear(void);

```

Constant	Value
NG_DBG_X	22
NG_DBG_Y_START	2
NG_DBG_PAL	4
NG_DBG_SUPPRESS_UNUSED	\

ng_demo_advanced

sdk/2d_engine/ng_demo_advanced.h — Prebuilt showcase routines used by the demo ROM.

```

void demo_smooth_camera(void);
void demo_palette_flash_cycle(void);
void demo_hitstop_and_shake(void);
void demo_particle_stress(void);
void demo_depth_starfield(void);
void demo_boss_depth_shrink(void);
void demo_perf_hud(void);

```

ng_feedback

sdk/2d_engine/ng_feedback.h — Hitstop, shake, palette flash and a sound hook, in one call.

ng_impact_event() is the whole game-feel package for a hit. Prefer it over assembling the four effects by hand so intensity stays consistent.

```
void ng_feedback_flash_red(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
uint8_t ng_feedback_is_slow_motion(void);
```

Constant	Value
NG_FEEDBACK_MAX_EVENTS	4
NG_IMPACT_LIGHT	0
NG_IMPACT_MEDIUM	1
NG_IMPACT_HEAVY	2
NG_IMPACT_BOSS	3

ng_particles

sdk/2d_engine/ng_particles.h — A 32-slot fixed pool with typed spawns and priority eviction.

The pool never grows and never allocates. When it is full a new spawn evicts the lowest-priority live particle, so a flood of dust cannot push out an explosion.

```
NGParticle * ng_spawn_dust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_slash_trail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_explosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_smoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle * ng_spawn_magic_spark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
uint8_t ng_particles_count_priority(uint8_t priority);
```

Constant	Value
NG_PART_MAX_PARTICLES	32
NG_PART_PRI_CRITICAL	0
NG_PART_PRI_NORMAL	1
NG_PART_PRI_OPTIONAL	2
NG_PART_DUST	0
NG_PART_HIT_SPARK	1
NG_PART_SLASH_TRAIL	2
NG_PART_EXPLOSION	3
NG_PART_MAGIC_SPARK	4
NG_PART_SMOKE	5
NG_PART_WATER_SPLASH	6
NG_PART_METAL_HIT	7
NG_PART_ANIM_FRAMES	4

PART 6

2D Engine Reference — C++

The C++14 build of the same engine: classes, methods, and how it relates to the C API.

`docs/API_2D_ENGINE_CPP.md`

****Eagle Software · Neo Geo SDK v1.7.0 · sdk/2d_engine_plus/****

The C++14 build of the 2D engine. This is not a different engine — it is the same design with the same public ABI, so a C game can link against this build and a C++ game can link against the C one.

```
make p1 USE_2D_PLUS=1      # link this engine
make p1                    # link the C engine
```

```
#include "sdk/2d_engine_plus/ng_engine_plus.hpp"
```

What this build adds

Every function in `API_2D_ENGINE_C.md` exists here unchanged, declared extern "C". On top of them this build adds member methods and singletons, so the same work reads as `cam.follow(...)` and `CharManager::instance()` instead of a free function plus an explicit pointer. The wrappers are inline and compile to the same hardware writes — there is no runtime cost and no vtable.

Compiled under `-std=c++14 -fno-exceptions -fno-rtti -fno-threadsafe-statics -ffreestanding -march=68000`. No standard library, no malloc, no floating point, and no global constructors that depend on initialisation order.

Choosing between the builds

Build	Choose it when
sdk/2d_engine	You are writing C. Smallest code-size footprint and the simplest call chain.
sdk/2d_engine_plus	You are writing C++. You want member methods, singletons, and the scene-stack helpers.

One engine per ROM. `games/demo_plus` (id 778) is the smoke-test target that links only against this build.

Classes and methods

Extracted from the headers by `tools/gen_api_reference.py`. Types with no methods are plain data structures shared with the C build.

ng_actions.hpp

Table-driven action scripts with sound and FX hooks.

****ActionRegistry****

```
static ActionRegistry& instance();
void NEOGEO_USER init();
void NEOGEO_USER registerScript(uint16_t action_id, const NGAActionCmd *script);
const NGAActionCmd* NEOGEO_USER get(uint16_t action_id);
void NEOGEO_USER setSoundHooks(NGActionSfxHook sfx, NGAActionMusicHook music);
void NEOGEO_USER setFxHook(NGActionFxHook fx);
void NEOGEO_USER update(struct NGCharacter *c);
NGActionSfxHook sfxHook();
NGActionMusicHook musicHook();
NGActionFxHook fxHook();
```

ng_camera.hpp

Follow, dead zone, look-ahead, shake, cinematic pan, bounds clamp.

****NGCamera****

```
void NEOGEO_USER init();
void NEOGEO_USER setBounds(int16_t left, int16_t top, int16_t right, int16_t bottom);
void NEOGEO_USER setFollowSpeed(uint8_t speed);
void NEOGEO_USER setDeadZone(uint8_t half_w, uint8_t half_h);
void NEOGEO_USER setLookAhead(int16_t max_x, int16_t max_y, uint8_t rate);
void NEOGEO_USER snap(int16_t x, int16_t y);
void NEOGEO_USER shake(uint8_t amp, uint8_t frames);
void NEOGEO_USER panTo(int16_t dest_x, int16_t dest_y, uint8_t speed);
void NEOGEO_USER update(int16_t target_x, int16_t target_y, int16_t target_vx);
void NEOGEO_USER apply(int16_t target_x, int16_t target_y, int16_t target_vx);
int16_t NEOGEO_USER worldToScreenX(int16_t world_x);
int16_t NEOGEO_USER worldToScreenY(int16_t world_y);
```

ng_chars.hpp

The character pool: 64 objects, movement, animation, hitboxes, damage.

****NGCharacter****

```
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER setSpeed(int16_t vx_px, int16_t vy_px);
void NEOGEO_USER setSpeedFp(int32_t vx_fp, int32_t vy_fp);
void NEOGEO_USER addSpeedFp(int32_t ax_fp, int32_t ay_fp);
void NEOGEO_USER setSprite(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBase, uint8_t palette);
void NEOGEO_USER setAssetBounds(uint16_t tileStart, uint16_t tileEnd);
uint8_t NEOGEO_USER bindAsset(const NGSpriteAssetView *asset);
void NEOGEO_USER setCullMargin(int16_t l, int16_t r, int16_t t, int16_t b);
void NEOGEO_USER setAnimClip(const NGAnimClip *clip);
void NEOGEO_USER animUpdate();
void NEOGEO_USER setTileStride(uint16_t stride);
void NEOGEO_USER setBody(int16_t x, int16_t y, int16_t w, int16_t h);
void NEOGEO_USER setPriority(uint8_t band, int16_t offset);
void NEOGEO_USER damage(uint8_t amount);
void NEOGEO_USER heal(uint8_t amount);
NGRect NEOGEO_USER bodyRect();
NGRect NEOGEO_USER hitRect();
```

****CharManager****

```
static CharManager& instance();
void NEOGEO_USER init();
NGCharacter* NEOGEO_USER add(uint8_t kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGCharacter *c);
void NEOGEO_USER clearKind(uint8_t kind);
void NEOGEO_USER resetSlot(uint8_t index);
void NEOGEO_USER clearArena(uint8_t arena_id);
void NEOGEO_USER setDefaultArena(uint8_t arena_id);
void NEOGEO_USER setFixedStep(uint8_t updates_per_frame);
void NEOGEO_USER updateFixed();
void NEOGEO_USER defragSlots();
NGCharacter* NEOGEO_USER find(uint8_t kind);
NGCharacter* NEOGEO_USER at(uint8_t index);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER indexOf(const NGCharacter *c);
void NEOGEO_USER setInterrupt(uint8_t kind, NGCharInterrupt fn);
void NEOGEO_USER update();
void NEOGEO_USER draw();
void NEOGEO_USER clearUploadSlot(uint8_t index);
```

ng_feedback.hpp

Hitstop, shake, palette flash and a sound hook, in one call.

****FeedbackSystem****

```
static FeedbackSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER setSfxHook(NGFeedbackSfxHook hook);
void NEOGEO_USER update();
void NEOGEO_USER hitstop(uint8_t frames);
uint8_t NEOGEO_USER isHitstop();
uint8_t NEOGEO_USER hitstopRemaining();
void NEOGEO_USER shake(NGCamera *cam, uint8_t amp, uint8_t frames);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t frames);
void NEOGEO_USER slowMotion(uint8_t frames);
uint8_t NEOGEO_USER isSlowMotion();
uint8_t NEOGEO_USER slowMotionRemaining();
void NEOGEO_USER impactEvent(uint8_t impact_kind, uint8_t palette_slot, const uint16_t *base_pal, NGCamera *cam, uint16_t sfx_id, int16_t spark_x, int16_t spark_y, uint16_t spark_tile, uint8_t spark_pal);
```

ng_npcs.hpp

NPC wrappers: patrol bounds, home position, think callbacks.

****NpcManager****

```
static NpcManager& instance();
void NEOGEO_USER init();
NGNpc* NEOGEO_USER spawn(uint8_t npc_kind, uint8_t char_kind, int16_t x, int16_t y);
void NEOGEO_USER remove(NGNpc *npc);
NGNpc* NEOGEO_USER at(uint8_t index);
NGNpc* NEOGEO_USER find(uint8_t npc_kind);
NGCharacter* NEOGEO_USER charOf(NGNpc *npc);
uint8_t NEOGEO_USER count();
void NEOGEO_USER setThink(NGNpc *npc, NGNpcThink think, uint16_t interval_frames);
void NEOGEO_USER setHome(NGNpc *npc, int16_t x, int16_t y);
void NEOGEO_USER setPatrolBounds(NGNpc *npc, int16_t min_x, int16_t max_x, int16_t min_y, int16_t max_y);
void NEOGEO_USER update();
static void NEOGEO_USER thinkPatrol(NGNpc *npc, NGCharacter *c);
static void NEOGEO_USER thinkHover(NGNpc *npc, NGCharacter *c);
```

ng_palette_fx.hpp

Fade, flash, pulse and colour cycle on a palette bank.

****PaletteFxSystem****

```
static PaletteFxSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
void NEOGEO_USER uploadBase(uint8_t palette_slot, const uint16_t *pal);
void NEOGEO_USER fadeIn(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER fadeOut(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashWhite(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashRed(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER flashBlue(uint8_t palette_slot, const uint16_t *base_pal, uint8_t duration);
void NEOGEO_USER pulse(uint8_t palette_slot, const uint16_t *base_pal, uint8_t period);
void NEOGEO_USER cycle(uint8_t palette_slot, const uint16_t *base_pal, uint8_t start, uint8_t end);
void NEOGEO_USER stop(uint8_t palette_slot);
uint8_t NEOGEO_USER active(uint8_t palette_slot);
```

ng_particles.hpp

A 32-slot fixed pool with typed spawns and priority eviction.

****ParticleSystem****

```
static ParticleSystem& instance();
void NEOGEO_USER init();
void NEOGEO_USER update();
uint16_t NEOGEO_USER draw(uint16_t first_slot, uint16_t sprite_budget_used);
NGParticle* NEOGEO_USER spawn(uint8_t type, uint8_t priority, int16_t x, int16_t y, int32_t vx_fp, int32_t vy_fp, uint8_t lifetime, uint16_t tile_base, uint8_t palette, uint8_t frame_count, uint8_t anim_period);
NGParticle* NEOGEO_USER spawnHitSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnDust(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSlashTrail(int16_t x, int16_t y, int8_t dir, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnExplosion(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnSmoke(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
NGParticle* NEOGEO_USER spawnMagicSpark(int16_t x, int16_t y, uint16_t tile_base, uint8_t palette);
uint8_t NEOGEO_USER count();
uint8_t NEOGEO_USER countPriority(uint8_t priority);
```

ng_physics.hpp

Gravity, drag, solid rectangles, and grounded tests.

****PhysicsWorld****

```
static PhysicsWorld& instance();
void NEOGEO_USER init();
void NEOGEO_USER attach(NGCharacter *c, uint16_t flags);
void NEOGEO_USER detach(NGCharacter *c);
NGPhysicsBody* NEOGEO_USER body(NGCharacter *c);
void NEOGEO_USER setGravity(NGCharacter *c, int32_t gravity_fp, int32_t max_fall_fp);
void NEOGEO_USER setDrag(NGCharacter *c, int32_t drag_x_fp, int32_t drag_y_fp);
void NEOGEO_USER clearSolids();
uint8_t NEOGEO_USER addSolid(int16_t x, int16_t y, int16_t w, int16_t h, uint16_t flags);
void NEOGEO_USER updatePre();
void NEOGEO_USER resolve();
uint8_t NEOGEO_USER isGrounded(NGCharacter *c);
```

ng_sprite_group.hpp

Dirty-flag sprite chains — write only what changed.

****NGSpriteGroup****

```

void NEOGEO_USER init(uint16_t firstSprite, uint8_t strips, uint8_t heightTiles, uint16_t tileBase, uint8_t palette);
void NEOGEO_USER markDirty(uint8_t flags);
void NEOGEO_USER flush();
void NEOGEO_USER setTileBase(uint16_t tileBase);
void NEOGEO_USER setTileStride(uint16_t tileStride);
void NEOGEO_USER setPalette(uint8_t palette);
void NEOGEO_USER setActiveRows(uint8_t rows);
void NEOGEO_USER setPos(int16_t x, int16_t y);
void NEOGEO_USER move(int16_t dx, int16_t dy);
void NEOGEO_USER setScale(uint8_t xScale, uint8_t yScale);
void NEOGEO_USER setFlip(uint8_t hflip, uint8_t vflip);
void NEOGEO_USER setAutoAnim(uint8_t aa4, uint8_t aa8);
void NEOGEO_USER setVisible(uint8_t v);
void NEOGEO_USER upload();
void NEOGEO_USER updateTransform();
void NEOGEO_USER hide();
static void NEOGEO_USER hideRange(uint16_t firstSprite, uint16_t count);
static void NEOGEO_USER hideVramBase(uint16_t spriteBase, uint16_t count);
static void NEOGEO_USER hideAll();
static void NEOGEO_USER initHardware(uint16_t transparentTile);

```

ng_sprite_window.hpp

Tracks the strips a variable-size sprite currently occupies.

****NGSpriteWindow****

```

void NEOGEO_USER init(uint8_t owner, uint16_t first_slot, uint8_t max_strips);
void NEOGEO_USER reset();
void NEOGEO_USER setCurrent(uint8_t current_strips);
void NEOGEO_USER setShape(uint8_t current_strips, uint8_t current_rows);
void NEOGEO_USER clear();
void NEOGEO_USER clearTail();
void NEOGEO_USER hide();

```

Free functions

The extern "C" surface is identical to the C build. See `API_2D_ENGINE_C.md` for the full list; the module layout, constants, and semantics all carry over unchanged.

PART 7

Sound Driver Reference

The Z80 driver, the YM2610, the command protocol, and the audio content pipelines.

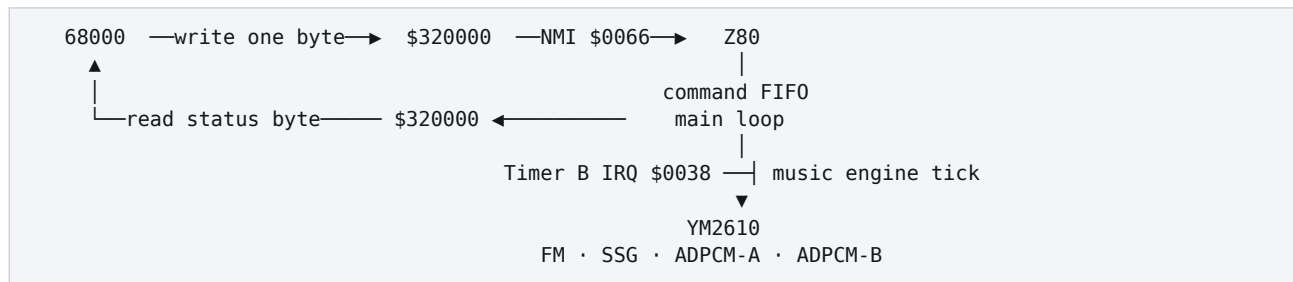
`docs/SOUND_DRIVER.md`

Eagle Software · Neo Geo SDK v1.7.0

Everything the Neo Geo makes noise with runs on a second CPU. This document explains the architecture, the 68000-side API you actually call, the raw command protocol underneath it, the content pipelines that feed it, and the practices that keep audio stable across scene changes.

The exhaustive per-function argument reference lives in `../sound/SOUND_DRIVER_GUIDE.txt`; this document is the map. For the composing tools see `SOUND_STUDIO_GUIDE.md`.

1. Architecture



1. The 68000 sends one-byte commands through `REG_SOUND`.
2. The Z80 receives each command by NMI at vector `$0066`.
3. The driver pushes commands into a FIFO in Z80 RAM.
4. The main Z80 loop drains the FIFO and dispatches.
5. Timer B IRQ at vector `$0038` advances the music engine.

Nothing about this is synchronous. `playSFX()` returns immediately; it has posted a byte, not played a sound. Design around that.

The four sound areas

Area	Channels	Use
ADPCM-A	6 hardware channels, one-shot	Short sound effects, voice samples
ADPCM-B	1 stream at a time	Backing TRACKs, ambience, long cues
SSG	3 square waves	Melody from the MML engine, plus a standalone SSG sequencer
FM	Compiled sequencer, 16-patch voice bank	Music, stings, formant effects

Where the code lives

File	Role
<code>sound/driver/driver.asm</code>	The Z80 driver. Command decoder, FIFO, music engine, patch loader.
<code>sound/driver/driver.c</code>	Experimental C rewrite of the same driver (make <code>m1rom-c</code>).
<code>sound/m1/m1.asm</code>	M1 ROM wrapper the driver is assembled into.
<code>sdk/neogeo.h</code> , <code>sdk/neogelib.c</code>	The 68000-side API — every function below.
<code>sdk/sound_ids.h</code>	Named track / SFX / FM / SSG ids.

File	Role
sdk/sound_voice_ids.h	Generated voice-sample ids (SOUND_VOICE_*).
sound/tools/*.py	MML, FM, SSG, patch, sample and V-ROM compilers.

2. The 68000-side API

Include `sdk/neogeo.h` and `sdk/sound_ids.h`. Call `soundInit()` once at boot before anything else.

Core control

Call	Effect
<code>soundInit()</code>	Boot the Z80 driver and YM2610. Required once.
<code>soundReset()</code>	Driver soft reset.
<code>soundStopAll()</code>	Stop every channel, every area.
<code>soundStopMusic()</code>	Stop music / SSG only; leaves SFX alone.
<code>soundCancelFade()</code>	Abort a fade and restore the stored base volumes.
<code>soundSceneReset()</code>	The one to call on a scene change — see §5.

Playback

Call	Plays
<code>playMusic(track)</code>	MML music track (SOUND_MUSIC_A..._H, 8 tracks)
<code>playSFX(n)</code>	ADPCM-A one-shot (SOUND_SFX_1..._12, plus the voice bank)
<code>playSFXB(n)</code>	ADPCM-B stream (SOUND_TRACK_*, 9 tracks; TRACK 4 is reserved for the eyecatcher)
<code>playFMTrack(n)</code>	Compiled FM track (SOUND_FM_A..._H, 8 tracks)
<code>playFMDebug()</code>	FM track 0 as a bring-up tone
<code>playSSGTrack(n)</code>	Standalone SSG sequence (SOUND_SSG_A..._I, 9 tracks)
<code>soundSetSSGPreset(n)</code>	Select one of 16 SSG timbre presets
<code>playInsertCoinSSG()</code>	The coin jingle

Scene helpers

`soundPlayDemoFM(fm_track)`, `soundPlayTitleMusic(music_track)` and `soundPlayGameLoop(music_track)` wrap the common "stop what's playing, start this, at the right volume" sequences. Prefer them over hand-rolled stop-then-start pairs.

Voice

The driver carries a generated speech bank — the letters a-z, digits, punctuation, and whole words — encoded as ADPCM-A samples.

Call	Effect
<code>speakText("READY!")</code>	Prefers a whole-word sample, then falls back to letters, digits and punctuation
<code>playVoiceWord(SOUND_VOICE_WORD_GAME_OVER)</code>	One whole-word sample
<code>playVoiceLetter(SOUND_VOICE_LETTER_A)</code>	One letter
<code>playVoiceNumber(n)</code>	A direct number sample when one exists, otherwise digit-by-digit
<code>playGetReadyVoice()</code> , <code>playAttackVoice()</code> , <code>playCoinThenReady()</code>	Prebuilt cues

`speakText()` calls `soundStopMusic()` first, because speech and SSG melody fight for the same channels.

Volume, tempo, fades

```

soundSetADPCMAVolume(v);    soundSetADPCMBVolume(v);
soundSetSSGVVolume(v);      soundSetFMVolume(v);        /* 0..15 */
soundSetTempo(t);           soundFMSetTempo(period);    /* 1..8 */
soundFadeOut();              soundFadeIn();
soundFadeOutSpeed(speed);    soundFadeInSpeed(speed);

```

Chip-level controls

Call	YM2610 register
<code>soundSetADPCMBPan(pan)</code>	\$11 L/R pan — \$C0 both, \$80 left, \$40 right, \$00 mute
<code>soundFMSetLFO(rateEn)</code>	\$22 LFO — bit 3 enables, bits 0-2 set the rate
<code>soundSetSSGNoise(period)</code>	\$06 noise period, 5 bits

The LFO register is global to the chip, so patch changes deliberately leave it alone: a `soundFMSetLFO()` setting survives every patch switch and note-on.

CSM — Composite Sine Mode

CSM auto-keys FM channel 2 from Timer A, which is how you get formant and vowel-slide effects out of an FM chip.

```

soundFMCSMBegin(period_hi);    /* driver cmd $1B — reg $24/$25/$27 */
soundFMCSMSweep(from, to, ms); /* slide the formant, then end CSM */
soundFMCSMEnd();               /* driver cmd $1C — reg $27 = $00 */

```

Pair CSM with an active `playFMTrack()` so channel 2 has a patch to be keyed with. `soundStopAll()` ends CSM too, so the mode cannot bleed into the next scene.

3. The raw command protocol

Use the SDK functions first. Reach for raw bytes only for a feature with no wrapper yet — `playSoundtest(index)` in `sdk/neogelb.c` sends one directly.

Byte	Meaning
\$01	Driver init

Byte	Meaning
\$02	BIOS eyecatcher / boot music (mapped to music track 1)
\$03	Driver soft reset
\$04	Stop all playback
\$05 <i>n</i>	ADPCM-A volume
\$06 <i>n</i>	ADPCM-B volume
\$07 <i>n</i>	SSG music volume
\$0A <i>n</i>	Fade-out speed
\$0C	Stop ADPCM-A only
\$0D	Stop ADPCM-B only
\$0E <i>n</i>	Live tempo divisor
\$0F	Stop music / SSG only
\$11	Cancel fade, restore base volumes
\$12 <i>n</i>	Fade-in speed
\$13 <i>n</i>	FM volume (low nibble, 0-15)
\$14 <i>n</i>	SSG preset index (low nibble, 0-15)
\$16 <i>n</i>	ADPCM-A sample by full index — used for sample ids ≥ 64
\$1B <i>n</i>	CSM begin, Timer-A high byte
\$1C	CSM end
\$20+ <i>n</i>	Start music track <i>n</i>
\$28, \$29	Direct trigger, ADPCM-B samples 0 and 1
\$30	FM debug tone (FM track 0)
\$31 <i>n</i>	FM track select
\$32 <i>n</i>	SSG track select
\$40+ <i>n</i>	ADPCM-A sample <i>n</i> , compact range 0-63
\$80+ <i>n</i>	ADPCM-B sample <i>n</i>

Multi-byte commands need spacing

A prefix byte and its parameter are two separate NMIs on the Z80. Sending them back to back from a tight 68000 loop can drop the second one. In the demo's sound and shooter chapters every multi-step setup is separated by a `waitVbl()`, and that is the pattern to copy:

```
soundSetSSGPreset(3);
waitVbl();
playSSGTrack(SOUND_SSG_B);
```

4. Content pipelines

`make_sound` runs all of these. Each has its own target if you only changed one kind of asset.

Source	Compiler	Target	Lands in
games/<game>/sound/mml/*.mml	mml_compile.py	mml	music_data.inc
sound/fm/*.fm	fm_compile.py	fm	fm_data.inc
sound/fm/patches.fm	fm_patch_compile.py	fmpatches	fm_patch_table.inc
sound/ssg/*	ssg_compile.py	ssg	ssg_data.inc
sound/ssg/ config	ssg_config_compile.py	ssgconfig	ssg_config.inc
sound/samples/in_wav_a/*.wav	adpcm_enc.py	samples	ADPCM-A, sample_table.inc
sound/samples/in_wav_b/*.wav	adpcm_enc.py	samples	ADPCM-B
sound/samples/in_wav_a_voice/*.wav	vrom.py	vrom	V-ROM + sdk/sound_voice_ids.h
all of the above	wla-z80 + wla-link	m1rom	<id>-m1.m1

Sample numbering is positional: `playSFX(0)` is 1.wav, `playSFX(SOUND_SFX_7)` is 7.wav. Renaming or inserting a file rennumbers everything after it.

The Python sample path is the default; `S0X=/path/to/sox` switches to the SoX conversion path only if you explicitly want it.

Two driver builds

```
make m1rom-asm      # assemble sound/driver/driver.asm (the shipping driver)
make m1rom-c       # build the experimental C driver
make compare-driver # build both and diff the ROM images
```

`driver.asm` is the driver that ships. The C version exists to be diffed against it, and `compare-driver` is how you check a change to one against the other.

5. Practices that keep audio stable

Initialise once. `soundInit()` in `game_boot()`, never per scene.

Reset on scene change, do not stack stops. `soundSceneReset()` exists because the failure mode of hand-rolled teardown is a fade left running, a CSM mode left set, or a volume left at zero, three scenes later.

Space multi-byte commands. See §3.

Do not poll for completion. There is no reliable "sample finished" handshake for general use. Drive timing from your own frame counter.

Fade, then stop. A hard `soundStopAll()` under a playing ADPCM-B stream is audible as a click. `soundFadeOut()` then stop reads as intentional.

Watch the ADPCM-A channel count. Six channels. A seventh trigger steals one of the six, and the one it steals is whichever the driver picks — not necessarily the least important. Budget effects the way you budget sprites.

6. What the demo shows

Chapter 23 of the demo ROM (SOUND) plays through the mix: MML music via `soundPlayGameLoop(SOUND_MUSIC_E)`, SFX triggers, FM, and SSG in sequence. Chapter 24 (SKY LANCE) drives a pure-SSG mix under gameplay, with every Z80 command spaced by `waitVbl()` so multi-step setup applies cleanly. See `DEMO_CHAPTERS.md`.

PART 8

Graphics Pipeline Reference

PNG to C-ROM: the quantiser, asset ordering, generated outputs, and Artbox Studio.

`docs/ARTBOX_PIPELINE.md`

Eagle Software · Neo Geo SDK v1.7.0

Artbox converts PNG sources into Neo Geo graphics data and the generated metadata your game code binds against. It is the stage that decides how your art actually looks on the machine, so it is worth understanding rather than treating as a black box.

The colour target

A Neo Geo colour word is not RGB444. Bit 15 is the "dark" bit, bits 14-12 carry the least-significant bit of each channel, and bits 11-0 carry the top four bits of R, G and B. The result is **5 bits per channel, 32 levels, in steps of 8**. Every colour the pipeline emits is snapped onto that lattice.

Each sprite or tile can use **15 colours plus transparent** — 4 bits of index, where index 0 is transparent.

The v1.7.0 quantiser

`artbox/img2neo_tile.py` is the default conversion path for every build; both `make art` (Linux) and `make -f MakefileWin32.mak art export ARTBOX_TILE=1` so this is the route every ROM takes.

The v1.7.0 pass rebuilt it around four ideas. Together they moved mean colour error (CIE-Lab dE) from 9.10 to 8.33 and p95 error from 18.19 to 15.66 across the reference asset set, while cutting measured dither speckle by a third.

Match in Lab, not in RGB. Palette selection, the k-means refinement, and the final per-pixel remap all run in CIE-Lab through a precomputed 32^3 lattice LUT (`lab_lattice_lut()`, `lab_palette()`). Matching in one space and measuring in another is what makes a quantiser that scores well look wrong; doing everything in Lab is what makes the numbers mean something. An earlier attempt that matched in luma-weighted YCbCr while measuring in Lab made sprites measurably *worse*.

Refit the palette onto the hardware grid, then re-optimize. k-means picks colours in continuous space, and snapping them to the 32-level lattice afterwards moves every centroid slightly — so the result is no longer optimal for the colours it was chosen for. `refine_palette_on_lattice()` runs Lloyd iterations with the snap *inside* the update step, keeps a move only when total error actually drops, and re-seeds duplicate or unowned slots from the worst-served pixels. Nothing is wasted on a colour no pixel wants.

Dither with blue noise, and only where dithering helps. `_void_and_cluster_mask()` builds a 32×32 void-and-cluster blue-noise mask (0.04 % of its energy in the low frequencies), used to order the mix between the two nearest palette entries along a serpentine scan. A dead band skips the mix entirely when a pixel sits within 18 % of either endpoint — those are the pixels where dithering adds visible speckle without adding colour. This is what removed the "noise dots" from flat regions.

Push contrast and saturation before clustering, with a soft knee. Fifteen colours across a whole image pull everything toward the middle of the gamut, because the average of a cluster is always less saturated than its members. `_neo_palette_pop()` compensates before clustering — a contrast blend and a saturation push, both stronger for sprites than for backgrounds — and compresses the top and bottom eighth with a tanh knee instead of clipping. Clipping folds every boosted highlight onto pure white, which throws colour detail away before k-means ever sees it.

Sprite and background paths are tuned separately: sprites get a wider smoothing cut and no ordered mixing (a dithered sprite edge reads as noise against a moving background), backgrounds get the full treatment.

Results are cached; `_VIVID_CACHE_VERSION` invalidates the cache when the algorithm changes.

Alternative pipelines

Route	Script	When
Default (tile-local)	img2neo_tile.py	Every build. Per-tile k-means++, per-tile dither, greedy MAE bank dedup, Lab-nearest remap.
CRT	img2neo_crt.py	make art-crt or ARTBOX_CRT=1. Lab k-means, horizontal-biased dither, gamma 1.20 / contrast 1.10 pre-boost.
HD	img2neo_hd.py	Opt-in. Bilateral filter, CLAHE on the Lab L channel, unsharp mask, blue-noise dither. Best for photographs.
Legacy	ARTBOX_LEGACY=1	The original nearest-neighbour path, kept for diffing.
FIX HD	fixtiles_hd.py	Per-tile palette FIX conversion; -sharp-text binarises glyph sources.

img2neo.py also exposes `alpha_bleed()`, applied after `fit_sprite_rgba` so anti-aliased contours stop baking the source PNG's hidden transparent-pixel RGB (usually near-white) into the indexed sprite. That is the fix for the pale halo around sprite edges.

Asset ordering and ids

Categories are processed in a fixed order:

```
backgrounds, characters, effects, eyecatcher, npc, screens, titles
```

then any other directory alphabetically. Within a category, files are processed in filename order.

An asset id is its 1-based position in that sequence. Inserting a file in the middle of a category rennumbers everything after it, and your game code refers to assets by id. The shipped games pad filenames to control placement, and you should too.

characters and npcs share **one derived 15-colour master palette per group** (`build_master_sprite_palettes()`); every other category gets per-asset palettes. A sprite that looks wrong after being moved between categories is almost always hitting this, not a conversion bug.

Rule matching in `assets.cfg` has three priorities; the lowest is a rule with an empty `match_category` plus a filename pattern, which is how the `legacy_screens` rule (`[0-9].png, mode = screen`) catches numbered files in any directory.

Generated outputs

File	Contents
<id>-c1.c1, <id>-c2.c2	Sprite tile data, the C-ROM pair
<id>-s1.s1	FIX tile ROM
screens.c	One <code>showScreenN()</code> entry point per asset
sprite_meta.h	Per-asset geometry — tile base, strips, rows, padding
assets_manifest.json	The manifest the rest of the pipeline reads
neo.pal, std.pal, neopal.bin	Palette data
infix_palettes.h	FIX palettes for INFIX photo pages

`main.c` in each game holds the generated `ng_screen_table[]` and `ng_screen_count`, which `ng_bg_set_by_id()` resolves screen ids through. A game with no screen art links against a weak fallback in `ng_bg.c` instead.

Verifying a build

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py
python3 artbox/validate_assets.py
```

`make art` runs the palette check itself and fails the build on a mismatch.

Artbox Studio

`artbox/artbox_studio.py` is the PyQt6 desktop front end for everything above. It does not replace the `make` targets — it drives the same scripts — but it makes the feedback loop visual.

Tab	What it does
Pipeline	Runs every step (<code>img2neo</code> , <code>genscreens</code> , <code>gen_sprite_meta</code> , <code>fixtiles</code> , <code>romdbimgimport</code> , <code>romtiles</code> , <code>romdbfiximport</code> , <code>createromdb</code>) with a status pill per step and a live log. "Run Full Pipeline" chains the standard set.
Asset Browser	Tree of every PNG under <code>artbox/in/*</code> and <code>artbox/infix/</code> , with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16×16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export <code>anim_frame_tile[]</code> / <code>anim_frame_dur[]</code> C arrays in vblank units.
Level Designer	Paint a 20×14 tilemap with tile thumbnails; export a <code>level_tile[LEVEL_H][LEVEL_W]</code> C array.
HD Compare	Run one PNG through both the standard and HD pipelines and see the indexed results side by side. Output lands in <code>artbox/out/_hd_cmp/</code> ; originals are untouched.
ROM Inventory	Every ROM kind (<code>p1</code> / <code>m1</code> / <code>s1</code> / <code>v1</code> / <code>c1</code> / <code>c2</code>) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for <code>artbox/assets.cfg</code> , with save and reload.

The current pipeline is rule-driven through `artbox/assets.cfg`.

Inputs

- `artbox/in/` contains PNG artwork for sprite pages and screen/background pages.
- `artbox/infix/` contains FIX-layer source artwork.

Rule Fields

Each `[rule:name]` section in `artbox/assets.cfg` can define:

- `pattern`: filename glob, such as `sprite_*.png`
- `mode`: screen or sprite

- **category**: gameplay category written into generated metadata
- **fit**: crop or pad
- **anchor**: placement anchor such as center or bottom-center
- **target_width / target_height**: output canvas size
- **palette** and image-processing settings such as dither, contrast, and kmeans_iters

Categories

Generated metadata uses these category constants:

```
#define NG_ASSET_CATEGORY_BACKGROUND 0
#define NG_ASSET_CATEGORY_MAIN_CHARACTER 1
#define NG_ASSET_CATEGORY_OPPONENT 2
#define NG_ASSET_CATEGORY_NPC 3
```

Current naming rules:

PNG pattern	Mode	Category
[0-9].png	screen	background
background_*.png	screen	background
zz_npc_forest_alley.png	screen	background
sprite_*.png	sprite	main_character
opponent_*.png	sprite	opponent
z_npc_*.png	sprite	npc

Generated Files

The pipeline writes:

- artbox/assets_manifest.json
- artbox/out.srt
- artbox/sprite_meta.h
- artbox/screens.c
- Neo Geo graphics ROM data consumed by the main build

sprite_meta.h exposes NGSpriteAssetMeta records with tile base, palette bank, sprite strips, active rows, content bounds, mode, and category.

Build

Run the full artbox pipeline:

```
cd artbox
./makeartbox.sh
```

Windows:

```
cd artbox
makeartbox.bat
```

If `makeartbox.bat` fails with `PIL/img2neo` not available, install packages through the same interpreter used by `py`:

```
py -0p
py -m pip --version
py -m pip install --upgrade pip
py -m pip install numpy pillow pypng
cd artbox
py -c "import sys; print(sys.executable); import PIL, numpy, png, img2neo; print('OK')"
```

Then rebuild the ROM:

```
make
```

Runtime Use

The 68000 demo includes `artbox/sprite_meta.h` and uses the generated metadata to place sprite frames without hardcoding each frame's tile offsets, palette bank, active rows, and visible strips.

Gameplay code can use `meta->category` to distinguish backgrounds, main character frames, opponents, and NPC frames while keeping the graphics ROM layout stable.

Artbox Studio UI

`artbox/artbox_studio.py` is a PyQt6 desktop tool for inspecting and editing the graphics ROM without running the full pipeline.

```
python3 artbox/artbox_studio.py
```

Tabs

Tab	Function
Tile Grid	Scrollable viewer of all tiles from the combined C-ROM. Zoom 1-6x, per-asset palette, click to select, jump to any asset.
Sprite Designer	Configure a multi-strip sprite: tile base, strips, active rows, stride, palette bank. Exports a ready-to-paste C <code>char_set_sprite</code> call.
Hitbox Editor	Place draggable body and hit rectangles over the sprite preview. Exports a C hitbox struct definition.
Pixel Paint	16-color pixel art editor for patching individual tiles directly in the C-ROM buffer. Saves back to <code>777-c1.c1</code> / <code>777-c2.c2</code> .

Tile decoding follows the exact layout written by `romtiles.py`:

- Each C-ROM stores **64 bytes per 16×16 tile** (4 blocks × 8 rows × 2 bytes).
- Block order: top-right, bottom-right, top-left, bottom-left.
- `777-c1.c1` is byte-flipped (`romts.sh /f`): row bytes are `[plane_a, plane_b]`.
- `777-c2.c2` similarly: `[plane_c, plane_d]`.
- Color index per pixel: `d<<3 | c<<2 | b<<1 | a` (4-bit, maps into `palette[0..15]`).

PART 9

Tools and Utilities

Every helper script that ships with the SDK - art conversion, sound encoding, verification, launchers, installers and the GIMP plug-ins.

`docs/TOOLS.md`

Eagle Software · Neo Geo SDK v1.7.0

Every helper script that ships with the SDK, what it does, and when you would run it by hand. Most of these are invoked for you by a make target — the reason to know them individually is debugging, one-off conversions, and building your own pipeline steps.

Anything under `artbox/` reads and writes the *current game's* `artbox` directory, which the makefiles select through `ARTBOX_DATA_DIR`. Run them through `make art` unless you specifically want to drive one stage on its own.

1. Graphics — `artbox/`

1.1 Conversion cores

These are the scripts that turn pixels into Neo Geo data. One of them is selected per build; the others are alternatives you opt into.

Script	Role			
<code>img2neo_tile.py</code>	The default. Tile-local quantiser: per-tile Lab k-means++, per-tile dither, hardware-lattice palette refit, void-and-cluster blue-noise dithering, greedy MAE bank dedup, Lab-nearest remap. Both makefiles export <code>ARTBOX_TILE=1</code> , so this is the route every ROM takes.			
<code>img2neo_crt.py</code>	CRT-preprocessing route: CIE-Lab k-means, horizontal-biased Floyd-Steinberg, gamma 1.20 / contrast 1.10 pre-boost. <code>make art-crt</code> or <code>ARTBOX_CRT=1</code> .			
<code>img2neo_hd.py</code>	High-quality photo route: bilateral filter, CLAHE on the Lab L channel, unsharp mask, blue-noise dither. CLI flags <code>--no-bilateral</code> , <code>--no-clahe</code> , <code>--no-unsharp</code> , <code>--dither blue</code>	<code>fs\</code>	<code>ordered\</code>	<code>none`</code>
<code>img2neo.py</code>	The original nearest-neighbour path, kept for diffing (<code>ARTBOX_LEGACY=1</code>). Also home to <code>alpha_bleed()</code> and <code>fit_sprite_rgba()</code> , which the other routes use.			
<code>fixtiles.py</code>	FIX-layer (S-ROM) tile conversion.			
<code>fixtiles_hd.py</code>	Per-tile-palette FIX conversion. <code>--sharp-text</code> binarises glyph and HUD sources; writes a sidecar <code>*.pal.json</code> with the per-tile palette table.			

1.2 Pipeline stages

Run in this order by `makeartbox.sh` / `makeartbox.bat`:

Script	What it does
<code>creatoromdb.py</code>	Creates the ROM database (<code>neorom.db</code>) the later stages import into.
<code>romdbimgimport.py</code>	Imports converted sprite and screen images into the ROM database. Owns the screen fitting modes — <code>crop</code> , <code>contain</code> , <code>pad</code> , <code>letterbox</code> — and records the real content box per asset.

Script	What it does
romdbfiximport.py	Imports FIX-layer tiles into the ROM database.
romtiles.py	Packs the database's tile data into the C-ROM pair (<id>-c1.c1, <id>-c2.c2).
genscreens.py	Emits screens.c — one showScreenN() entry point per asset, plus the geometry each needs.
gen_sprite_meta.py	Emits sprite_meta.h from assets_manifest.json — per-asset tile base, strips, rows and padding.
gen_eyecatcher.py	Emits eyecatcher.c from the manifest entries with category == "eyecatcher", as an animation.
gen_assets_cfg.py	Regenerates assets.cfg with the default per-category rules on each make art.
genmapdb.py, genmapfile.py	Build the tile map database and the map file the ROM tooling consumes.
asset_rules.py	The rule engine behind assets.cfg: category matching, filename patterns, fit and anchor resolution. Imported by the stages above rather than run directly.

1.3 Utilities

Script	Use
romtool.py	A portable Python 3 port of ROMWak. Splits and interleaves ROM images — /b splits alternating bytes, and the byte-swap and pad operations the P-ROM build relies on.
validate_assets.py	Checks generated sprite_meta.h for tile-window safety — that every asset's declared window is inside its allocated tile range. python3 artbox/validate_assets.py games/demo/artbox/sprite_meta.h
count_assets.py	Prints the number of assets the current in/ tree will produce. Useful before an id-sensitive change, because an asset id is its position in the build order.
fix_sprite_alpha.py	Repairs sprite transparency before C1/C2 generation, for sources whose alpha channel does not survive an editor round-trip. Run from the repository root.
process_eyecatcher_ng.py	Pre-quantises eyecatcher PNGs onto the Neo Geo colour grid before the main conversion, so the k-means stage starts from colours the hardware can actually hold.
sync_main_screens.py	Syncs the generated showScreenN() declarations and ng_screen_table[] into a game's main.c.
artbox_studio.py	The PyQt6 desktop front end — see §5.

1.4 Shell wrappers

Script	Platform	What it runs
makeartbox.sh / makeartbox.bat	Linux / Windows	The full art build for one game. GAME_ID=777 ./artbox/makeartbox.sh demo
artbox.sh / artbox.bat	both	The bare stage sequence, without the per-game path setup.

Script	Platform	What it runs
romts.sh / romts.bat	both	Sprite-tile ROM packing.
romfx.sh / romfx.bat	both	FIX ROM packing.
makeclean.sh / makeclean.bat, clean.sh / clean.bat	both	Remove generated artbox intermediates.

1.5 GIMP plug-ins — [tools/gimp-plugins/](#)

Manual pre-processing for source art, before it reaches `img2neo*.py`. Two sets of the same three tools: `gimp2.10/` (Python-Fu, `gimpfu`) and `gimp3.2/` (GObject-Introspection, GEGL buffers).

Plug-in	What it does
smart_background_remove	Removes a flat or near-flat background while keeping anti-aliased contours intact.
remove_white_edge_noise	Strips the pale fringe that survives a background removal — the same class of artefact <code>alpha_bleed()</code> handles in the pipeline, but applied at source.
neogeo_hd_pixel_enhance	Sharpens and cleans up-scaled or photographic source so the quantiser has crisper edges to cluster.

2. Sound — [sound/tools/](#)

2.1 Compilers

Script	Input	Output
mml_compile.py	*.mml music tracks	music_data.inc. Handles the SSG/music MML directives, including @aN to fire ADPCM-A sample <i>N</i> from a track.
fm_compiler.py	*.fm FM tracks	fm_data.inc
fm_patch_compiler.py	sound/fm/patches.fm	fm_patch_table.inc — 31-byte patches over four operators
ssg_compile.py	SSG tracks	ssg_data.inc
ssg_config_compile.py	SSG preset config	ssg_config.inc — 6-byte presets
vrom.py	encoded ADPCM banks	the V-ROM, sample_table.inc, and the generated <code>sdk/sound_voice_ids.h</code> . Bundles the voice bank immediately after the SFX bank and exports <code>ADPCMA_VOICE_BASE / ADPCMA_VOICE_COUNT</code> .

2.2 Sample encoding

Script	What it does
adpcm_enc.py	The YM2610 ADPCM encoder, matching real hardware decoding exactly. ADPCM-A is OKI-style 4-bit at a fixed 18.5 kHz with a 12-bit signed accumulator; ADPCM-B is the streamed format.
wav_to_raw_pcm.py	WAV to raw PCM, the stage before ADPCM encoding. Handles 8-, 16-, 24- and 32-bit sources.
enc_wave16le_a.sh / .bat	WAV to 16-bit little-endian PCM for the ADPCM-A bank.
enc_wave16le_b.sh / .bat	The same for the ADPCM-B bank.
enc_wave16le_a_voice.sh / .bat	The same for the voice bank. Skips cleanly when in_wav_a_voice/ is absent.
adpcm_enc_a.sh / .bat	PCM to ADPCM-A.
adpcm_enc_b.sh / .bat	PCM to ADPCM-B.
adpcm_enc_a_voice.sh / .bat	PCM to ADPCM-A for the voice bank.
adpcm_enc_process.sh / .bat	Chains all three encode stages in order: SFX, voice, then ADPCM-B.

The Python path is the default on both platforms. Set `S0X=/path/to/sox` only when you specifically want the SoX conversion route.

2.3 Driver build and comparison

Script	What it does
m1rom.sh / m1rom.bat	Assembles and links the M1 (Z80 driver) ROM with <code>wla-z80 + wlalink</code> .
vrom.sh / vrom.bat	Shell wrappers around <code>vrom.py</code> .
combine_split_driver.py	Merges the fixed Z80 prelude (<code>driver_prelude.asm</code>) with compiler-generated driver assembly. Used only by the experimental C driver path.
compare_m1.py	Compares two generated M1 ROM images and reports the first mismatch, with SHA256 of each. This is what <code>compare-driver</code> uses to check the assembly and C drivers against each other.

2.4 Sound Studio

`sound/sound_studio.py` — the PyQt6 desktop front end. See §5.

3. Verification — tools/

Run by the build, and worth running by hand when something looks wrong.

Script	Checks
<code>check_game_cfg.py</code>	That <code>GAME=</code> matches <code>CURRENT_GAME</code> in the selected <code>game.cfg</code> . Every <code>make</code> target depends on this, and it is what stops one game's assets being built into another game's ROM.
<code>verify_artbox_palettes.py</code>	Per-game artbox palette output: manifest count, palette bank range, and <code>neopal.bin</code> record integrity (136 bytes per record — an int32 index plus sixteen 64-bit palette words). <code>make art</code> runs it and fails the build on a mismatch.
<code>verify_sfix_output.py</code>	The generated FIX ROM: size, format, and tile layout.
<code>gen_api_reference.py</code>	Regenerates <code>docs/API_2D_ENGINE_C.md</code> and <code>docs/API_2D_ENGINE_CPP.md</code> from the engine headers, so the API reference cannot drift from the code. Run it after adding or changing a public engine call.
<code>make_manual_pdf.py</code>	Builds <code>docs/neogeosdk_v1.7.0_manual.pdf</code> — the programmer's manual, the SDK library reference, and both 2D engine references — from the markdown in <code>docs/</code> . Run <code>gen_api_reference.py</code> first if the engine changed.

```
python3 tools/verify_artbox_palettes.py --root . --game demo
python3 tools/verify_sfix_output.py --root .
python3 tools/gen_api_reference.py
python3 tools/make_manual_pdf.py
```

4. Launchers and environment

4.1 Repository root

Script	What it does
<code>neogeosdk.sh</code> / <code>neogeosdk.bat</code>	The interactive build menu — pick a game and a target without remembering the flags. Same as <code>make menu</code> .
<code>run.sh</code> / <code>run.bat</code>	Build and run the current game in MAME.
<code>run_neogeosdk.sh</code> / <code>.bat</code>	Run the built ROM in MAME with the correct rompath, hashpath and BIOS.
<code>run_neogeosdk_debug.sh</code> / <code>.bat</code>	The same with the MAME debugger attached.
<code>run_neogeosdk_lua.bat</code>	Run with a MAME Lua <code>-autoboot_script</code> , for automated inspection and input driving.
<code>run_neogeosdk_trace.bat</code>	Run with MAME tracing enabled.
<code>set_sdkhome.bat</code>	Sets <code>SDKHOME</code> for a Windows shell session.

4.2 dist/

make dist produces per-game run scripts alongside the packaged ROM: run_demo.sh / .bat, run_helloworld.sh / .bat, run_game.sh / .bat, and a _debug variant of each.

4.3 install/

One-shot toolchain installers:

Script	Host
install-linux.sh	Generic Linux
install-ubuntu.sh	Ubuntu, with the distro package names resolved
install-windows.bat	Native Windows
install-wsl.bat	WSL, including the Windows-side setup

See ../install/README.md.

4.4 win/

Prebuilt Windows helper binaries used by MakefileWin32.mak so the Windows build needs no extra installs: srec_cat.exe, srec_cmp.exe, srec_info.exe, xxd.exe.

5. Desktop applications

Both are PyQt6 and both drive the same scripts the makefiles do — they make the feedback loop visual, they do not replace the build.

5.1 Artbox Studio — artbox/artbox_studio.py

Tab	What it does
Pipeline	Runs every conversion step with a status pill per step and a live log. "Run Full Pipeline" chains the standard set; HD variants are opt-in.
Asset Browser	Tree of every PNG under artbox/in/* and artbox/infix/, with dimensions, file size, and a native-scale preview.
Hex Sprite Inspector	Pick a tile index and see its decoded 16 × 16 grid, the raw C1/C2 ROM bytes, and a live palette-swap dropdown. The tool for debugging palette assignment and tile order.
Movement Designer	Build a frame sequence (tile + duration), loop-preview it at a chosen FPS, and export anim_frame_tile[] / anim_frame_dur[] C arrays in vblank units.
Level Designer	Paint a 20 × 14 tilemap with tile thumbnails; export a level_tile[LEVEL_H][LEVEL_W] C array.
HD Compare	Run one PNG through both the standard and HD pipelines and compare the indexed results. Output lands in artbox/out/_hd_cmp/; originals are untouched.
ROM Inventory	Every ROM kind (p1 / m1 / s1 / v1 / c1 / c2) for every game folder, with size, mtime, and present/absent colour coding.
Asset Rules	Editor for artbox/assets.cfg, with save and reload.

5.2 Sound Studio — `sound/sound_studio.py`

Track, Mix and ROM tabs, a live waveform view, and an MML designer for composing and auditioning music without a full `make_sound` cycle.

Full guide: `SOUND_STUDIO_GUIDE.md`.

6. Debug artefacts

`make debug-artifacts` writes these into `dump/`, using the cross-toolchain's own `binutils`:

File	From	Use
size report	<code>m68k-unknown-elf-size</code>	Section sizes — is the ROM about to overflow
symbol table	<code>m68k-unknown-elf-nm -n</code>	Symbol addresses, which is how you find a variable to watch from a MAME Lua script
ELF headers	<code>m68k-unknown-elf-readelf</code>	Segment layout
linker map	<code>ld -Map</code>	Where every object landed
disassemble mbly	<code>m68k-unknown-elf-objdump -d</code>	What the compiler actually emitted

`make gdb-trace` runs a batch GDB script and writes `dump/gdb_trace.txt`. See `GDB_GUIDE.md`.